

TEMA 3

PROGRAMACIÓN EN JAVASCRIPT

APLICACIONES WEB - GIS - CURSO 2021/22



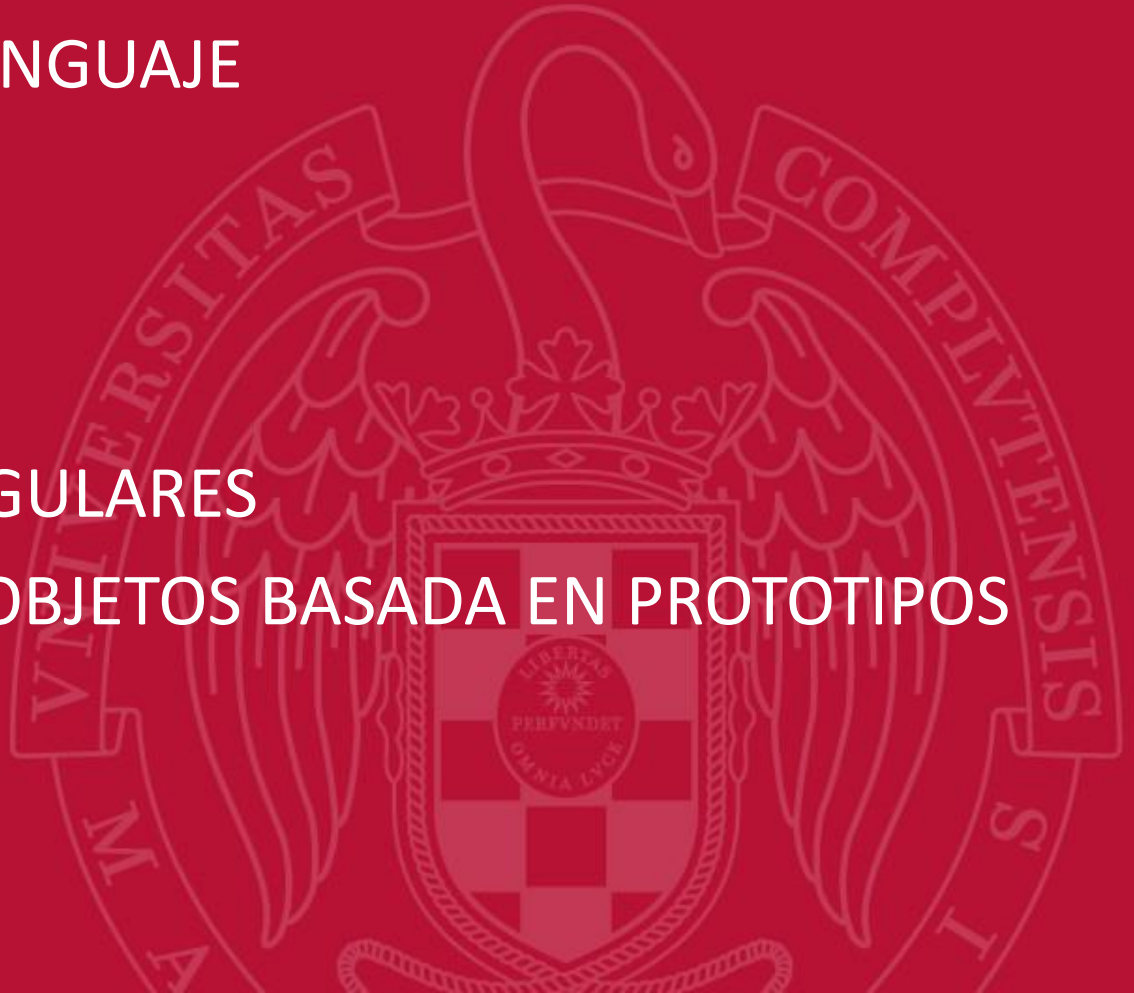
Esta obra está bajo una
Licencia CC BY-NC-SA 4.0 Internacional.

Jesús Cáceres Tello
Dpto de Sistemas Informáticos y Computación
Facultad de Informática
Universidad Complutense de Madrid

Este documento está basado en los materiales de **Manuel Montenegro** y **Marina de la Cruz**
bajo una licencia CC BY-NC-SA 4.0 Internacional

CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



EL LENGUAJE JAVASCRIPT

Javascript fue creado por Brendan Eich en 1995, para ser incluido en el navegador *Netscape*.

Netscape colaboraba en aquel momento con la empresa *Sun Microsystems*, propietaria por entonces del lenguaje Java.

Concebido inicialmente como un **lenguaje «pegamento»**, destinado a integrar los distintos componentes de las páginas web: applets, plugins, etc.

Pero su destino fue bien distinto...

HITOS EN EL JAVASCRIPT

- 1997 - **HTML Dinámico**

Los programas modifican dinámicamente la estructura de un documento HTML mediante la manipulación de su DOM.

- 2005 - **AJAX**

Los programas pueden realizar peticiones al servidor desde Javascript, lo que impulsó el paradigma de aplicaciones web de una sola página (SPA).

- 2009 - **Node.js**

Permite utilizar Javascript en el lado del servidor.

JAVASCRIPT Y ECMASCRIPT

En el año 1996 Netscape decidió estandarizar Javascript.

El estándar fue publicado por la organización *Ecma International*. El nombre del estándar era **ECMAScript**.

La versión actual del estándar (8ª edición) es ECMAScript 2017 y fue publicada en junio de 2017.

ECMA-260	Data Interchange on 356 mm Optical Disk Cartridges - WORM, using Phase Change Technology Capacity: 14,8 and 25 Gbytes per Cartridge (June 1997)	ISO/IEC 15898
ECMA-261	Broadband Private Integrated Services Network (B-PISN) - Service Description - Broadband Connection Oriented Bearer Services (B-PISN) (June 1997)	ISO/IEC 15899
ECMA-262	ECMAScript® 2017 Language Specification, 8 th edition (June 2017) <i>This edition has been prepared under the Ecma RF patent policy.</i>	ISO/IEC 16262 *
ECMA-263	Private Integrated Services Network (PISN) - Specification Functional Model and Information Flows - Call Priority Interruption and Call Priority Interruption Protection Supplementary Services (CPI(P)SD), 3 rd edition (December 2001)	ISO/IEC 15991 ETSI EN 301 655
ECMA-264	Private Integrated Services Network (PISN) - Inter-Exchange Signalling Protocol - Call Priority Interruption and Call Priority Interruption Protection Supplementary Services (QSIG-CPI(P)), 3 rd edition (December 2001)	ISO/IEC 15992 ETSI EN 301 656

JAVASCRIPT EN EL NAVEGADOR

Los principales navegadores contienen un **intérprete** que permite ejecutar los programas Javascript incluidos en las páginas web.

El componente del navegador encargado de esto recibe el nombre de **motor Javascript**.

Motores Javascript más conocidos:

- **SpiderMonkey**, utilizado en Firefox.
- **V8**, utilizado en Chrome.
- **Chakra**, utilizado en Edge.

NODE.JS

Es un intérprete del lenguaje Javascript, pensado para ejecutarse **fuera de un navegador**.

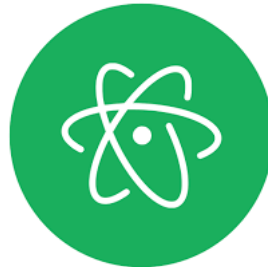
Su implementación está basada en el motor **V8** de Chrome.

Se utiliza principalmente para implementar las funcionalidades del lado del servidor en aplicaciones web.



JAVASCRIPT MÁS ALLÁ DE LA WEB

- **Aplicaciones de escritorio**, mediante Node y Electron. Ejemplos: Atom, Visual Studio Code, etc.
- **Aplicaciones móviles**, mediante Apache Cordova.
- **Extensiones de entornos de escritorio**, como GNOME Shell o Windows 10.



APACHE
CORDOVA™

EJEMPLO

```
// planets.js
// -----

"use strict";

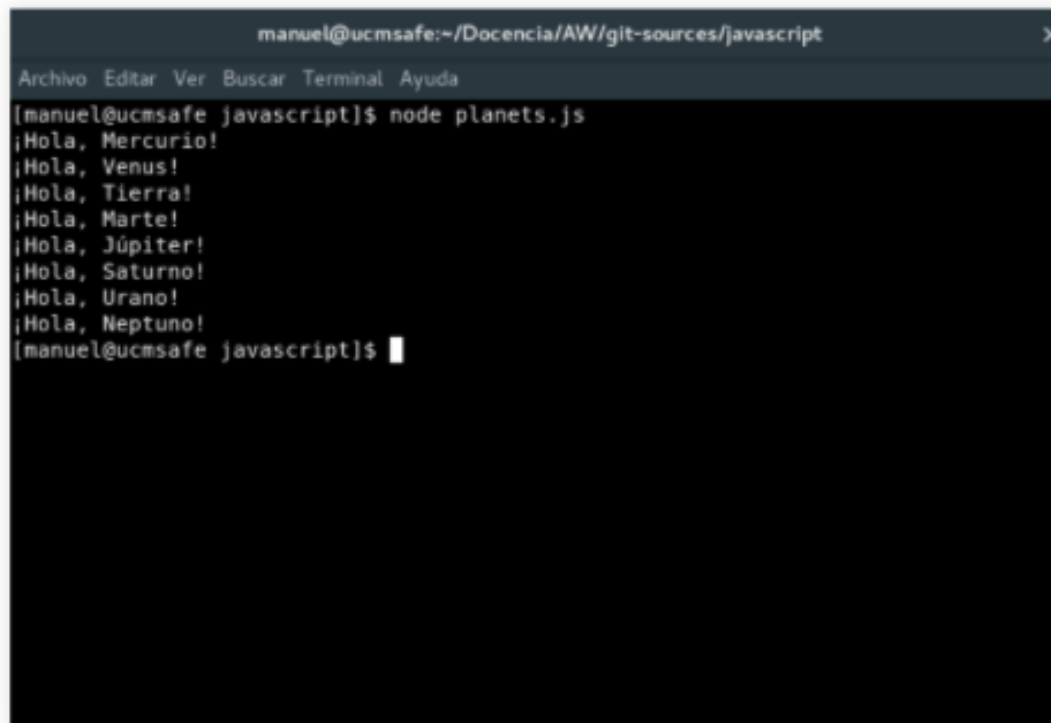
let planetas = [
  "Mercurio", "Venus", "Tierra",
  "Marte", "Júpiter", "Saturno",
  "Urano", "Neptuno"
];

planetas.forEach(p => {
  console.log(`¡Hola, ${p}!`);
});
```

EJEMPLO: EJECUCIÓN DESDE NODE

Si se tiene Node instalado, basta con ejecutar desde una línea de comandos:

```
node planets.js
```



A screenshot of a terminal window with a dark background. The title bar at the top reads "manuel@ucmsafe:~/Docencia/AW/git-sources/javascript" and includes a close button. Below the title bar is a menu bar with the options "Archivo", "Editar", "Ver", "Buscar", "Terminal", and "Ayuda". The terminal shows the command `[manuel@ucmsafe javascript]$ node planets.js` being executed. The output consists of eight lines, each starting with "¡Hola," followed by the name of a planet: "Mercurio!", "Venus!", "Tierra!", "Marte!", "Júpiter!", "Saturno!", "Urano!", and "Neptuno!". The prompt `[manuel@ucmsafe javascript]$` is visible at the bottom of the terminal, followed by a cursor.

```
manuel@ucmsafe:~/Docencia/AW/git-sources/javascript
Archivo Editar Ver Buscar Terminal Ayuda
[manuel@ucmsafe javascript]$ node planets.js
¡Hola, Mercurio!
¡Hola, Venus!
¡Hola, Tierra!
¡Hola, Marte!
¡Hola, Júpiter!
¡Hola, Saturno!
¡Hola, Urano!
¡Hola, Neptuno!
[manuel@ucmsafe javascript]$
```

EJEMPLO: EJECUCIÓN DESDE NAVEGADOR

Si, alternativamente, se quiere ejecutar un programa desde el navegador, ha de importarse el fichero *Javascript* desde un documento HTML, y abrir éste último desde el navegador.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <script src="planets.js" type="text/javascript"></script>
  </head>
  <body>
    Abre las herramientas del desarrollador y recarga la página
    para visualizar el resultado del programa.
  </body>
</html>
```

EJEMPLO: EJECUCIÓN DESDE NAVEGADOR (II)

Una vez cargada la página, abrir la consola de Javascript.

En Firefox: *Desarrollador* → *Consola Web* (Ctrl + Mayús + K)

En Chrome: *Herramientas del desarrollador* → *Consola*



EJEMPLO (II)

Fichero *bucle.html*

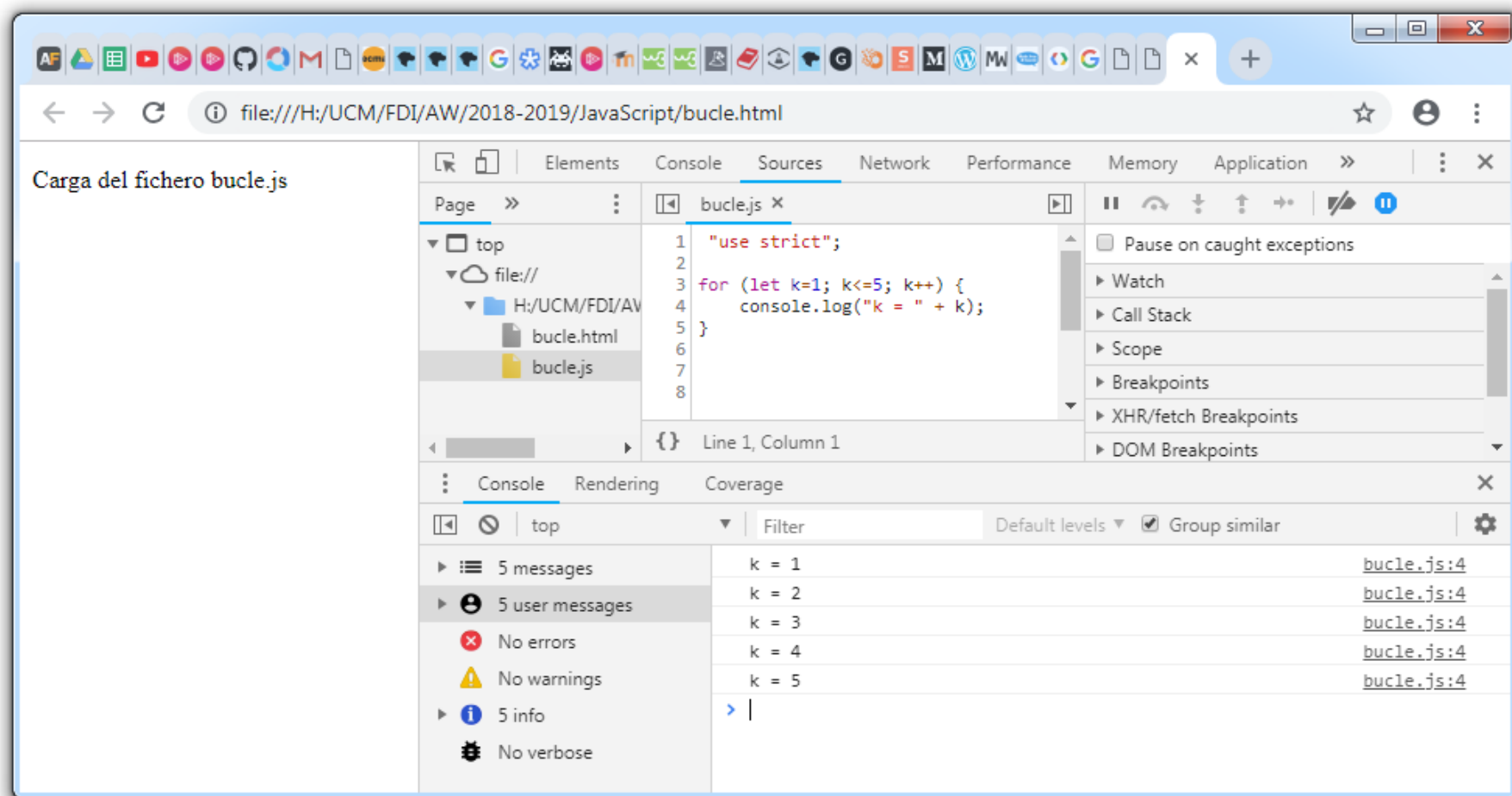
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <script src="bucle.js" type="text/javascript"></script>
  </head>
  <body>
    Carga del fichero bucle.js.
  </body>
</html>
```

Fichero *bucle.js*

```
"use strict";

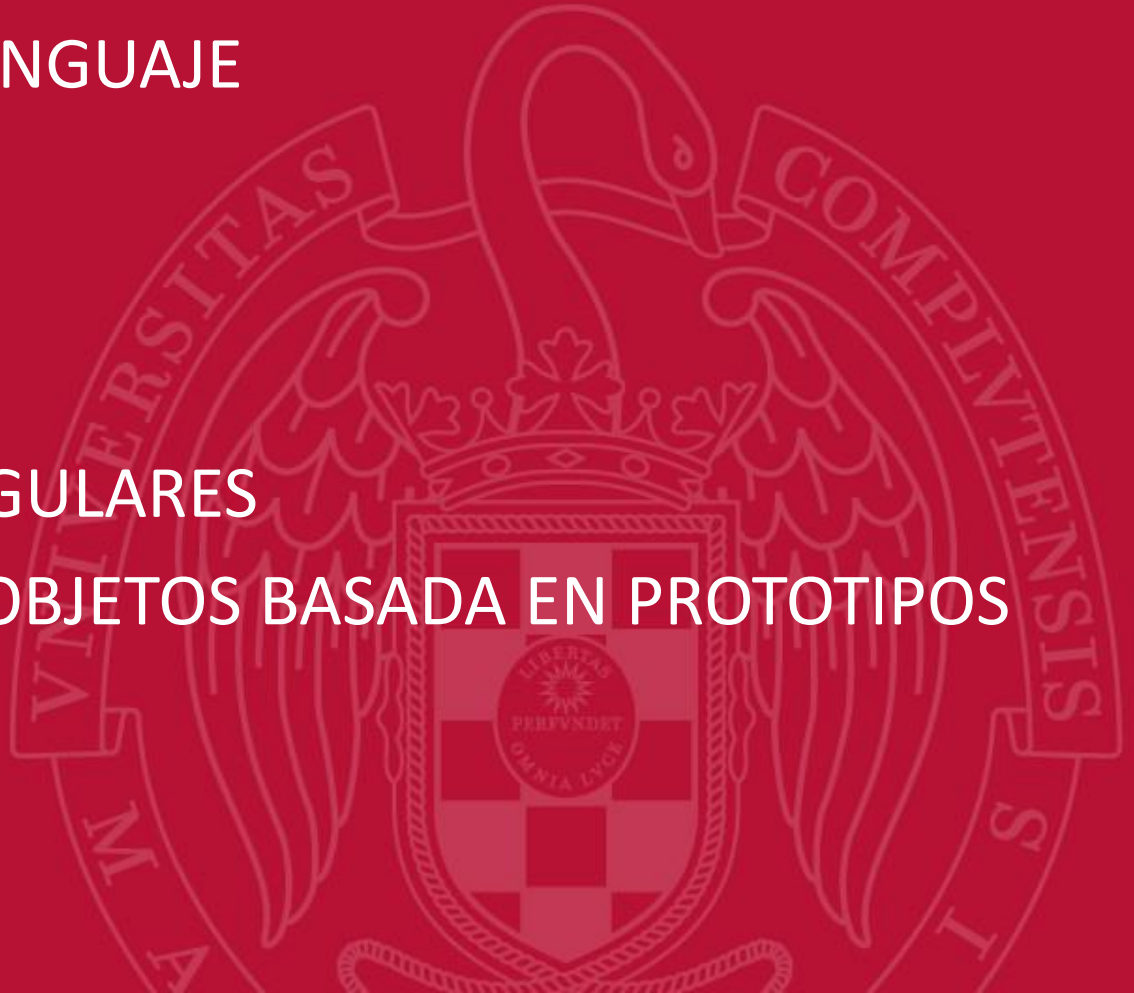
for (let k=1; k<=5; k++) {
  console.log("k = " + k);
}
```

DEPURACIÓN EN NAVEGADOR



CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



¿¿“USE STRICT” ??

Javascript es un lenguaje muy flexible.

...pero esto conlleva una gran responsabilidad.

Javascript tiene algunas características que hacen que el lenguaje sea, a veces, **demasiado permisivo**.

Esto hace que programar en Javascript fuese una tarea propensa a errores.

¿¿“USE STRICT” ??

Introduciendo la cadena `"use strict"` al principio del programa hace que éste se evalúe en **modo estricto**.

Este modo hace que el compilador detecte y prohíba el uso de características demasiado permisivas del lenguaje:

- Utilizar variables sin declararlas.
- Borrar una variable mediante `delete`.
- Duplicidades en nombres de atributos y parámetros.
- Estructuras poco recomendables: `with`.

```
var obj = { hello: "world" },
    value;
function without_with () {
  for ( var i = 0; i < 9999; i++ ) {
    value = obj.hello;
  }
}
function with_with() {
  with(obj) {
    for ( var i = 0; i < 9999; i++ ) {
      value = hello;
    }
  }
}
```

Función	Llamadas	%	Tiempo	Avg
with_with	10	91.08%	96.549ms	9.655ms
without_with	10	8.92%	9.453ms	0.945ms

```
function foo() { "use strict"; with({}); }
```

ASPECTOS GENERALES DEL JAVASCRIPT

- Sensible a mayúsculas y minúsculas
- Las sentencias se terminan con punto y coma (no es obligatorio).
- Nombre de identificadores (variables, funciones, ...)
 - Pueden contener letras, dígitos, _ y \$
 - No pueden comenzar por un dígito
- Declaración de variables y asignaciones

```
let x = 3;  
let y = x + 1;  
y += 3;
```

- Comentarios:

```
// Comentario de una línea  
/* Comentario de varias  
líneas */
```

Declaración de variables: Ámbitos

- Global
 - Navegador: objeto window
 - Fuera del navegador: depende del entorno
- Local: función
- Bloque: {} a partir de ES2015/ES6

Declaración con las palabras reservadas:

- var
- let y const (desde ES2015)

DECLARACIÓN DE VARIABLES

Declaración de variables **globales** con **var** y **let**

```
var x;  
var x,y;  
var x=8;  
var x=8, y=9;  
var x, y=9;  
let z=20;  
...
```

Declaración de variables **locales** con **var** y **let**

```
function foo() {  
  var k=100;  
  let j=89;  
  console.log(k);  
  console.log(j);  
}  
...
```

Declaración de variables **de bloque** con **var** y **let**

```
...  
{  
  var x=9;  
  let z=20;  
}  
...
```

Declaración de constantes con **const**

```
...  
const MAXIMO=1000;  
...
```

DECLARACIÓN DE VARIABLES

Se permite el uso de una variable declarada con **var** antes de su declaración.

```
x=8;           // Uso de antes de la declaración
console.log(x); // Imprime 8
var x;         // Declaración
```

Este comportamiento es posible por el proceso de **"hoisting"**: mover las declaraciones de las variables al principio de su ámbito.

```
var x;         // Declaración
x=8;           // Uso de antes de la declaración
console.log(x); // Imprime 8
```

COMPARATIVA ENTRE var Y let

- El **hoisting** sólo se hace sobre variables **var** en cualquier ámbito.

```
console.log(x);    // Imprime undefined
var x=8;           // Declaración
console.log(y);    // ReferenceError: y is not defined
let y=9;           // Declaración
```

- Ámbito **global**: sólo las **var** pertenecen al objeto global.

```
var x=8;
let y=9;
console.log(window.x);    // Imprime 8
console.log(window.y);    // Imprime undefined
```

- Ámbito **local**: ambas trabajan igual.

REDECLARACIONES

- Con **var** se permite en cualquier ámbito (cuidado con los bloques).

```
var x=8;  
// aquí x vale 8  
var x=1;  
// aquí x vale 1
```

```
var x=8;  
// aquí x vale 8  
{  
  var x=1;  
  // aquí x vale 1  
}  
// aquí x vale 1
```

- Con **let** NO se permite en ningún ámbito.

```
let y=9;  
  
let y=2;    // Identifier 'y' has already been declared
```

Siempre que sea posible utilizar **let**.

SENTENCIAS CONDICIONALES

- `if-then-else`

```
if(x < y) {  
    console.log(`${x} es menor que ${y}`);  
}  
else {  
    console.log(`${x} NO es menor que ${y}`);  
}
```

- `switch`

```
switch(day) {  
    case 6:  
    case 7:  
        console.log("Es fin de semana :-)");  
        break;  
    default:  
        console.log("Es día laborable :-(");  
}
```

BUCLES

- while

```
let resultado = 0;  
let numero = 100;  
let i = 0;  
  
while(i ≤ numero) {  
    resultado += i;  
    i++;  
}
```

- do-while

```
let resultado = 0;  
let numero = 100;  
let i = 0;  
  
do{  
    resultado += i;  
    i++;  
} while(i <≤ numero)
```

BUCLES (II)

- Bucles for:

```
for (let i = 0; i < b.length; i++) {  
    sum += b[i] + c[i];  
}
```

- Bucles for...of para recorrer arrays:

```
let arr = [4, 6, 10];  
let sum = 0;  
for (let x of arr) {  
    sum += x;  
}
```

BUCLES (III)

- break y continue

```
let i = 0;
let x = "fichero.pdf"

while (i < x.length) {
  if (x[i] === ".") break;
  console.log(x[i]);
  i++;
}
```

```
let x = [1,2,3,4,5];
let z = 1;

for (let i = 0; i < x.length; i++) {
  if (x[i] % 2 === 0)
    continue;
  console.log(x[i]);
  z *= x[i];
}

console.log(z);
```

FUNCIONES

- Definición de funciones:

```
function abs(x) {  
  if (x < 0) {  
    return x;  
  } else {  
    return -x;  
  }  
}
```

- Llamadas a funciones y métodos:

```
x = abs(-3);  
console.log(`El valor absoluto de x es ${x}`);
```

MANEJO DE EXCEPCIONES

- Bloques try-catch o try-catch-finally:

```
try {  
    funcion_no_existe();  
} catch (e) {  
    console.error(e.message);  
} finally {  
    console.log("Esto se ejecuta siempre");  
}
```

No se especifica el tipo de excepción

- Lanzamiento de excepciones:

```
throw new Error("Fichero no encontrado");
```

- Atributos de **Error**:
 - **Error.message**: mensaje de error.
 - **Error.stack**: pila de ejecución.
 - **Error.name**: nombre de la clase del error.

CADENAS DE TEXTO

- Inicialización:

```
let str = "Esto es una cadena";
```

o bien

```
let str = 'Esto es una cadena';
```

- Acceso al carácter i-ésimo:

```
str[2]  // → "t"
```

- Métodos:

```
str.slice(2, 5);  
    // → "to es"  
" vale ".trim();  
    // → "vale"  
str.split(" ");  
    // → ["Esto", "es",  
    //     "una", "cadena"]  
str.toUpperCase();  
    // → "ESTO ES UNA CADENA"  
str.toLocaleUpperCase();  
    // → "ESTO ES UNA CADENA"  
str.startsWith("Est");  
    // → true  
"ab".repeat(5);  
    // → "ababababab"
```

CADENAS PLANTILLA

Si delimitamos una cadena entre comillas invertidas (```), podemos utilizar la sintaxis `${...}` para introducir expresiones Javascript en su contenido.

```
let nombre = "Araceli";
let edad = 27;
let cadena = `Me llamo ${nombre} y tengo ${edad} años`;
console.log(cadena);
// → Me llamo Araceli y tengo 27 años

console.log(`Pero el año que viene tendré ${edad + 1} años`);
// → Pero el año que viene tendré 28 años
```


OPERADORES

- Relacionales: `==`, `===`, `!=`, `!==`, `<`, `<=`, `>`, `>=`
- Aritméticos: `+`, `-`, `*`, `/`, `%`
- Lógicos: `&&`, `||`, `!`
- A nivel de bit: `&`, `|`, `^`, `>>`, `<<`, `>>>`

```
var num = 0;  
var obj = new String("0");  
var str = "0";  
var b = false;
```

```
console.log(num == num); // true  
console.log(obj == obj); // true  
console.log(str == str); // true
```

```
console.log(num == obj); // true  
console.log(num == str); // true  
console.log(obj == str); // true  
console.log(null == undefined); // true
```

```
// both false, except in rare cases  
console.log(obj == null);  
console.log(obj == undefined);
```

```
console.log(num === num); // true  
console.log(obj === obj); // true  
console.log(str === str); // true
```

```
console.log(num === obj); // false  
console.log(num === str); // false  
console.log(obj === str); // false  
console.log(null === undefined); // false  
console.log(obj === null); // false  
console.log(obj === undefined); // false
```

ARRAY

- Inicialización:

```
let x = [4, 6, "pepe", 1, 3];
```

Corchetes en lugar de {}

```
let z = [];
```

Vector vacío

```
let m = new Array(3);
```

- Acceso:

```
console.log(x[3]); // → 1  
m[2] = "Elemento nuevo";
```

- Longitud:

```
x.length // → 5  
z.length // → 0  
m.length // → 3
```

OBJETOS

- Inicialización:

```
let x = {  
  nombre: "Nicolás",  
  apellidos: "Ortega",  
  edad: 14  
};  
  
let y = {};    // objeto vacío
```

- Acceso:

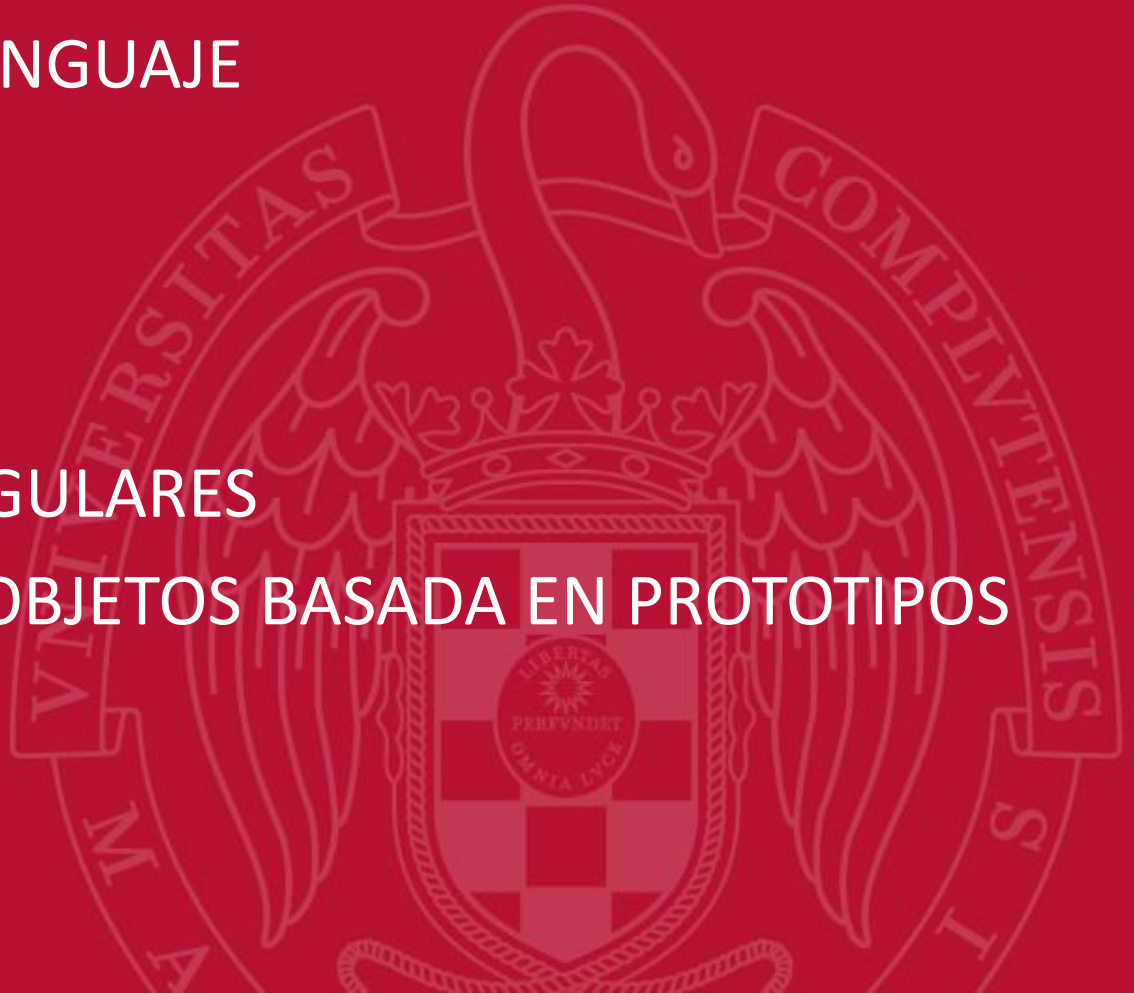
```
console.log(x.nombre);    // → Nicolás  
console.log(x["nombre"]); // → Nicolás
```

- Modificación:

```
x.edad++;
```

CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



ONCE RAREZAS DE JAVASCRIPT

(para alguien que viene de Java)

1. Javascript es dinámicamente tipado.
2. Los objetos no son necesariamente instancias de clases.
3. Valores indefinidos, nulos y NaN.
4. Las conversiones invisibles.
5. Argumentos que sobran y faltan.
6. Las funciones son ciudadanos de primera clase.
7. Las clases llegaron veinte años tarde.
8. La herencia basada en prototipos.
9. Los arrays son flexibles, y también son objetos.
10. Las funciones de orden superior sobre arrays.
11. Los módulos aún están por aterrizar.

ONCE RAREZAS DE JAVASCRIPT

RAREZA 1

**JAVASCRIPT ES UN LENGUAJE
DINÁMICAMENTE TIPADO**

DINÁMICAMENTE TIPADO

- **Lenguajes sin tipos**

No controlan los tipos de variables que declaran, es posible usar variables Muy poco usuales: TCL, ensamblador, etc.

- **Lenguajes estáticamente tipados**

Exigen saber el tipo de dato de una variable en el proceso de desarrollo. No permite que una variable tome valores que no son del tipo declarado inicialmente. Ejemplos: C, C++, Java, Go, Haskell, etc...

- **Lenguajes dinámicamente tipados**

No exigen saber el tipo de la variable, asigna el tipo cuando se le asigna el valor. Las variables pueden tomar valores de distintos tipos. La mayoría son lenguajes interpretados Ejemplos: Python, Ruby, **JavaScript**, PHP, etc...

- **Lenguajes fuertemente tipado:**

No permite violaciones de los tipos de datos, dado el valor de una variable de un tipo, no se puede usar como si fuera de otro tipo distinto, a menos que haya una conversión. Ejemplos: Ada, Modula 2, Fortran, Perl 6, Pascal, etc...

LENGUAJES ESTÁTICAMENTE TIPADOS

LENGUAJES ESTÁTICAMENTE TIPADOS

- Se detecta **en tiempo de compilación** que las operaciones se realizan sobre argumentos del tipo correcto.

Por ejemplo, el compilador avisa de errores como:

```
"pepe" * 24
```

- Estos lenguajes pueden requerir declaraciones de tipos por parte del programador.

Por ejemplo: Java, C, etc.

- En algunos lenguajes el compilador **infiere** los tipos.

Por ejemplo: Haskell, C++, etc.

LENGUAJE DINÁMICAMENTE TIPADO

- Se comprueba **durante la ejecución del programa** que las operaciones se realizan sobre argumentos de tipo correcto.

Por ejemplo, en Javascript:

```
if (...) {  
    y = 3 * "foo";  
}
```

El error debido a la expresión `3 * "foo"` solamente se manifestará si la condición del `if` se cumple.

DECLARACIÓN DE TIPO

En Java es necesario declarar el tipo de una variable antes de su primer uso:

```
int x;  
String z = "It's something";  
List<Integer> lista;
```

En Javascript no se indica el tipo de la variable a declarar. Se utilizan las palabras **let**, **const** o **var**.

```
let x;  
const z = "It's something";  
let lista;
```

¿Qué ventajas e inconvenientes tiene indicar el tipo de las variables en el programa?

TIPOS DISPONIBLES EN JAVASCRIPT

- `number`: tipo numérico (no hay distinción entre enteros y coma flotante).
- `boolean`: tipo booleano (incluye los valores `true` y `false`).
- `string`: tipo cadena.
- `undefined`: valor no definido.
- `function`: funciones.
- `object`: tipo objeto (incluye también arrays, expresiones regulares, etc).

VALORES INDEFINIDOS, NULOS Y NaN

EL VALOR INDEFINIDO (`undefined`)

Se utiliza para las variables no inicializadas y para atributos no existentes dentro de objetos.

```
let coordenadas = { x: 5, y: 6 };  
let v;  
console.log(v);           // → undefined  
console.log(coordenadas.z); // → undefined
```

EL VALOR NULO (`null`)

Se utiliza para denotar una referencia a objeto nula.

```
let x = null; // La variable 'x' esta inicializada, pero a una  
              // referencia nula.  
console.log(x); // → null
```

VALORES INDEFINIDOS, NULOS Y NaN (II)

EL VALOR NOT-A-NUMBER (NaN)

Se devuelve como resultado de operaciones aritméticas incorrectas:

```
Math.log(-2)    // → NaN  
parseInt("x2d") // → NaN
```

LOS VALORES INFINITOS

Infinity y *-Infinity* se utilizan para desbordamientos, o para operaciones que devuelven $\pm\infty$

```
Math.pow(2, 10000) // → Infinity  
Math.log(0)        // → -Infinity
```

VALORES INDEFINIDOS, NULOS Y NaN (III)

Se devuelve como resultado de operaciones aritméticas incorrectas:

```
Math.log(-2)    // → NaN  
parseInt("x2d") // → NaN
```

¡Cuidado con las comparaciones con NaN!

```
Math.log(-3) === NaN // → false  
NaN === NaN          // → false
```

Si se quiere determinar si una operación ha dado NaN como resultado, debe utilizarse la función `isNaN`

```
isNaN(NaN)           // → true  
isNaN(Math.log(-3))  // → true
```

VALORES INDEFINIDOS, NULOS Y NaN (III)

¡Cuidado con las comparaciones con NaN!

```
Math.log(-3) === NaN // → false  
NaN === NaN          // → false
```

Si se quiere determinar si una operación ha dado NaN como resultado, debe utilizarse la función `isNaN`

```
isNaN(NaN)           // → true  
isNaN(Math.log(-3)) // → true
```

COMPROBACIÓN DE TIPOS

La función **typeof** permite obtener el tipo de un elemento.
Devuelve una cadena con el nombre del tipo.

```
"use strict";

var x=10;
console.log(typeof(x));    // number
x = 1.23;
console.log(typeof(x));    // number

x = true;
console.log(typeof(x));    // boolean
x = false;
console.log(typeof(x));    // boolean
console.log(typeof(3<4));  // boolean

x = "cadena";
console.log(typeof(x));    // string

console.log(typeof(z));    // undefined

// continúa ...
```


COMPROBACIÓN DE TIPOS (II)

```
function suma(x,y) {  
    return x+y;  
}  
console.log(typeof(suma)); // function  
  
x = {alto:2, ancho:10};  
console.log(typeof(x)); // object  
x = [1,2,3]  
console.log(typeof(x)); // object  
x = new Date();  
console.log(typeof(x)); // object  
  
console.log(typeof(null)); // object
```

La función `typeof` no permite distinguir entre un objeto, un array, el valor `null`, etc.

La función `instanceof` si permite esa distinción.

CONVERSIONES DE TIPOS (I)

¿A qué valor se evalúan las siguientes expresiones?

```
"3" * 4           // → 12
3 * 4             // → 12
"3" * "4"         // → 12
"3" * "pepe"      // → NaN
"12" + "20"       // → 1220
"12" + 20          // → 1220
12 + "20"         // → 1220
Math.log10("1000") // → 3
"10" < "2"        // → true
"10" < 2           // → false
```

CONVERSIONES DE TIPOS (II)

¿En qué casos se cumple la condición del `if`?

```
if (23) { .... }           // → se cumple
if (-1) { .... }           // → se cumple
if (0) { .... }            // → no se cumple
if ("Pepe") { .... }       // → se cumple
if ("") { .... }           // → no se cumple
if ([1, 3]) { .... }        // → se cumple
if ([]) { .... }            // → se cumple
if (null) { .... }          // → no se cumple
if (undefined) { .... }     // → no se cumple
```

CÓMO EVITAR CONFUSIONES EN LOS TIPOS

Con este panorama, hay dos alternativas:

1. Aprenderse concienzudamente las reglas de conversión de JavaScript:

Información:

<http://webreflection.blogspot.com.es/2010/10/javascript-coercion-demystified.html>

2. **[Recomendado]** Hacer las conversiones explícitamente (si se duda del tipo de una expresión).

Funciones `Number(...)`, `String(...)`, `Boolean(...)`

FUNCIONES DE CONVERSIÓN (I)

La función `Number()`

```
Number("32")           // → 32
Number("2f3")           // → NaN
Number(true)            // → 1
Number(false)           // → 0
Number(undefined)       // → NaN
Number(null)            // → 0
Number(new Date())      // → 1476191814528 (depende de fecha y hora)
```

Cuando la función `Number` se llama sobre un objeto `x`, la conversión se aplica sobre `x.valueOf()`.

FUNCIONES DE CONVERSIÓN (II)

La función `parseInt()`

```
var a = parseInt("10") + "<br>";           // 10
var b = parseInt("10.00") + "<br>";         // 10
var c = parseInt("10.33") + "<br>";         // 10
var d = parseInt("34 45 66") + "<br>";      // 34 (se detiene en el espacio)
var e = parseInt(" 60 ") + "<br>";          // 60 (ignora espacios)
var f = parseInt("40 years") + "<br>";      // 40 (se detiene en el espacio)
var g = parseInt("He was 40") + "<br>";     // NaN (no empieza con número)
var h = parseInt("10", 10) + "<br>";        // 10 (base 10)
var i = parseInt("010") + "<br>";           // 10 (podía ser octal pero aquí es decimal)
var j = parseInt("10", 8) + "<br>";         // 8 (base 8 → 1*8 + 0)
var k = parseInt("0x10") + "<br>";         // 16 (hexadecimal)
var l = parseInt("10", 16) + "<br>";       // 16 (base 16 → 1*16 + 0)
```

Esta función parsea un String y devuelve un entero

FUNCIONES DE CONVERSIÓN (III)

La función `String()`

```
String(true)           // "true"
String(undefined)      // "undefined"
String(32)              // "32"
String(new Date())     // "Mon Oct 13 2025 10:25:30 GMT+0200
                       // (Central European Summer Time)"
```

Esta función aplicada sobre un objeto `x` llama al famoso método **`x.toString()`**

FUNCIONES DE CONVERSIÓN (IV)

La función `Boolean()`

```
Boolean("")           // false
Boolean(undefined)    // false
Boolean(null)         // false
Boolean(34)           // true
```

Los valores **false** son los siguientes:

Valor	Tipo	Cuándo se produce
false	Boolean	Literalmente el valor booleano false.
0	Number	Cero numérico exacto.
-0	Number	Cero negativo (resultado de divisiones como -1/Infinity).
0n	BigInt	Versión BigInt de cero (BigInt(0) o 0n).
""	String	Cadena vacía, sin ningún carácter.
null	Object (valor nulo)	Cuando una variable está intencionalmente vacía o sin objeto asignado.
undefined	Undefined	Cuando una variable se declara pero no se inicializa, o una función no retorna valor.
NaN	Number	Resultado de una operación numérica inválida (parseInt("x"), 0/0, etc.).

Los valores **true** son todos los demás.

LOS OPERADORES DE COMPARACIÓN

- `==` compara sólo valores.
- `===` compara valores y tipos.

```
"25" == 25      // → true
"25" === 25     // → false
false == 0      // → true
"" == 0         // → true
2.0 === 2       // → true (recuerda: no se distingue entre tipo
                  //      de enteros y de coma flotante)
```

Cuando se compara un string con un número, se convierte el string a número. Si el string es vacío se convierte a 0. Si el string no es numérico se convierte a `NaN`

Los operadores de comparación `!=` y `!==` trabajan de manera similar a `==` y `===`

LAS CONVERSIONES INVISIBLES

MORALEJA

Utiliza siempre `===` y `!==`

TIPOS PRIMITIVOS VS. OBJETOS

JavaScript es un lenguaje basado en objetos. Cada variable será de tipo **primitivo** o de tipo **Object**.

- **Tipos primitivos**
 - Numérico
 - Booleano
 - Cadena
 - Indefinido
 - Nulo
- **Tipo Object**
 - Objetos
 - Funciones

TIPOS PRIMITIVOS: INMUTABLES

Los tipos primitivos son **inmutables**

```
let str1 = "Cadena";  
let str2 = str1.slice(0, 3);  
console.log(str1); // → Cadena1  
console.log(str2); // → Cad
```

No modifica la cadena str1

Las comparaciones entre tipos primitivos se hacen por **valor**, no por referencia

```
let str1 = "Cadena";  
let str2 = "Cadena";  
let str3 = "Otra cadena";  
str1 == str2; // → true  
str1 === str2; // → true  
str1 != str3 // → true  
str1 !== str3 // → true
```

En Java `str1 == str2` devuelve **false**, pues son objetos distintos. En cambio, `str1.equals(str2)` devuelve **true**.

LA FUNCIÓN instanceof

La función `instanceof` permite conocer si un objeto pertenece a una clase. Devuelve `true/false`.

```
x = [1,2,3]
console.log(x instanceof Array);    // true

x = new Date();
console.log(x instanceof Date);     // true

x = 14;
console.log(x instanceof Array);    // false
console.log(x instanceof Date);     // false
```

Todos los objetos heredan de `Object`.

```
x = [1,2,3]
console.log(x instanceof Object);    // true

x = new Date();
console.log(x instanceof Object);    // true

x = 14;
console.log(x instanceof Object);    // false
```

EJEMPLO: PERTENENCIA A UNA CLASE

`x` ?

elemento	<code>typeof</code>	primitivo/Object
8	"number"	primitivo
true	"boolean"	primitivo
"cadena"	"string"	primitivo
var x	"undefined"	primitivo
null	"object"	primitivo
NaN	"number"	primitivo
function fun(...) {...}	"function"	Object
[1,2,3]	"object"	Object (Array)
{x:1, y:2}	"object"	Object

CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



OBJETOS EN JAVA

OBJETOS EN JAVA

En Java, un objeto es una **instancia de una clase**.

```
class Persona {  
    public String nombre;  
    public String apellidos;  
    public int edad;  
}  
...  
p = new Persona();
```

La clase es la que define los atributos y métodos del objeto.

```
p.nombre = "Ana María";  
System.out.println(p.edad);  
p.poblacion = "Barcelona";
```

¡Error! Atributo no declarado en clase

OBJETOS EN JAVASCRIPT

OBJETOS EN JAVASCRIPT

Un **objeto** en Javascript no es más que una colección de **atributos**, cada uno de ellos asociado a un **valor**.

```
let x = {  
  nombre: "Ana María",  
  apellidos: "Gamboa Esteban",  
  edad: 54  
};
```

Aquí se definen los atributos **al crear el objeto**.

El literal **{}** representa un objeto vacío (sin atributos)

```
let y = {};
```

CREACIÓN DE OBJETOS

El acceso a los atributos de un objeto se realiza mediante:

- El operador punto (.), igual que en Java.

```
x.apellidos // → "Gamboa Esteban"
```

- o bien, mediante el operador corchete

```
x["apellidos"] // → "Gamboa Esteban"
```

```
let atrib = "nombre";  
x[atrib] // → "Ana María"
```

El acceso a una propiedad inexistente devuelve **undefined**

```
x.noexiste // → undefined  
y.nombre // → undefined
```

ATRIBUTOS (I)

Modificación de atributos:

```
x.edad = x.edad + 1; // o bien: x.edad++  
x["nombre"] = "Ana Josefa";
```

Es posible añadir atributos sobre la marcha:

```
x.direccion = "Calle Bautista, 25";  
y.nombre = "Javier";  
  
console.log(x);  
// { nombre: 'Ana Josefa', apellidos: 'Gamboa Esteban', edad: 55,  
//   direccion: "Calle Bautista, 25" }  
console.log(y);  
// { nombre: 'Javier' }
```

...y también borrarlos:

```
delete x.edad;  
console.log(x);  
// { nombre: 'Ana Josefa', apellidos: 'Gamboa Esteban' }
```

ATRIBUTOS (II)

Los nombres de atributos no han de ser necesariamente identificadores válidos de Javascript. En caso de no serlo, han de aparecer entre comillas en la declaración:

```
let z = {  
  "Atributo con espacios": 21,  
  "14": "foo",  
  "false": "ok"  
};
```

Para acceder a estos atributos solo se puede utilizar la notación corchete

```
z["Atributo con espacios"] = 22;
```

LA FUNCIÓN `Object.keys()`

La función `Object.keys()` devuelve un array con los nombres de propiedades de un objeto:

```
let x = {  
  nombre: "Ana María",  
  apellidos: "Gamboa Esteban",  
  edad: 54  
};  
  
console.log(Object.keys(x));  
// [ 'nombre', 'apellidos', 'edad' ]
```

El operador `in` permite determinar la existencia de un atributo dentro de un objeto:

```
if ("edad" in x) {  
  console.log("x tiene un atributo llamado 'edad'");  
}
```

IGUALDAD DE OBJETOS

Cuando se aplica el operador `==` o `===` sobre objetos, se comprueba que las referencias a ambos lados del operador apuntan al mismo objeto.

```
let coord1 = { x: 20, y: 30 };  
let coord2 = { x: 20, y: 30 };  
let coord3 = coord1;  
  
console.log(coord1 === coord2); // → false  
console.log(coord1 === coord3); // → true
```

CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



DEFINICIÓN DE UNA FUNCIÓN

```
function imprime_args(p1, p2, p3) {  
  console.log(`p1: ${p1}`);  
  console.log(`p2: ${p2}`);  
  console.log(`p3: ${p3}`);  
}
```

Observar en la cabecera de la función que no se especifica nada acerca de los tipos de los parámetros ni del tipo de retorno de la función.

LLAMADA A UNA FUNCIÓN

```
imprime_args(1, "bar", true);
```

Resultado:

```
p1: 1  
p2: bar  
p3: true
```


CARACTERÍSTICAS DE LAS FUNCIONES

- Las funciones pueden asignarse a variables.
- Las funciones pueden pasarse como argumentos.
- Las funciones pueden devolverse como resultado de otras funciones.

ASIGNACIÓN DE UNA FUNCIÓN A UNA VARIABLE

Partimos de las siguientes definiciones:

```
function incrementar(x) {  
    return x + 1;  
}  
  
function duplicar(x) {  
    return 2 * x;  
}  
  
function cuadrado(y) {  
    return y * y;  
}  
  
function factorial(n) {  
    if (n <= 0) {  
        return 1;  
    } else {  
        return n * factorial(n - 1);  
    }  
}
```

ASIGNACIÓN DE UNA FUNCIÓN A UNA VARIABLE

Asignamos algunas de estas funciones a variables:

```
let i = incrementar;  
console.log(i(5));  
// Imprime: 6  
  
let f = factorial;  
console.log(f(10));  
// Imprime: 3628800
```

¡Cuidado con los paréntesis!

```
let i = incrementar(); // INCORRECTO  
// Esto realiza la llamada incrementar(undefined), y asigna el  
// valor resultante (que también es undefined) a i.  
  
console.log(i(5));  
// ERROR: i no es una función
```

FUNCIONES COMO PARÁMETROS DE FUNCIONES

La siguiente función recibe un array de funciones y un valor. Aplica cada una de las funciones del array al valor dado y muestra los resultados por consola:

```
function aplicar_funciones(funs, z) {  
  for (let i = 0; i < funs.length; i++) {  
    console.log(  
      `Aplicar función ${i} pasando ${z}: ${funs[i](z)}`  
    );  
  }  
}
```

Ejemplo:

```
aplicar_funciones([incrementar, duplicar, cuadrado, factorial], 5);
```

```
Aplicar función 0 pasando 5: 6  
Aplicar función 1 pasando 5: 10  
Aplicar función 2 pasando 5: 25  
Aplicar función 3 pasando 5: 120
```

FUNCIONES COMO RETORNO DE FUNCIONES

De igual modo, se puede devolver una función como resultado:

```
function buscar_por_nombre(nombre) {  
  switch(nombre) {  
    case "INC": return incrementar;  
    case "DUP": return dup;  
    case "SQR": return cuadrado;  
    case "FCT": return factorial;  
  }  
  // Si la función termina sin alcanzar un return,  
  // se considera que devuelve undefined  
}
```

Ejemplo:

```
var g = buscar_por_nombre("INC");  
console.log(g(10));
```

FUNCIONES COMO EXPRESIONES (I)

Se puede declarar una función utilizando una expresión.

En estos casos es posible omitir el nombre de la función
(**función anónima**)

```
let f = function() { console.log("Hola"); };  
f();  
  
let g = function(x, y) { return x + y; };  
console.log(g(3, 5));
```

Puede no ser una función anónima, pero no está disponible fuera de la función.

```
let g = function suma(x, y) { return x + y; };  
console.log(suma(3, 5));    // ReferenceError: suma is not defined
```

FUNCIONES COMO EXPRESIONES (II)

Dar un nombre a una función definida como expresión permite la escritura de funciones recursivas.

```
var factorial = function fac(n) {return n<2 ? 1 : n*fac(n-1)};
```

FUNCIONES COMO EXPRESIONES (III)

En el ejemplo anterior:

```
aplicar_funciones(  
  [ function(x) { return x - 3; },  
    function(x) { return Math.sqrt(x); },  
    factorial,  
    function(z) { return Math.log(z); } ], 2);
```

```
Aplicar función 0 pasando 2: -1  
Aplicar función 1 pasando 2: 1.4142135623730951  
Aplicar función 2 pasando 2: 2  
Aplicar función 3 pasando 2: 0.6931471805599453
```

¿Puede reemplazarse la referencia a **factorial** por otra función anónima?

HOISTING DE FUNCIONES

El hoisting se aplica a las declaraciones de las variables (**var**) y de las funciones.

NO se aplica a las funciones declaradas como expresiones.

```
"use strict";

let s;
let y;
let inc;

s = suma(6,7); // Se ejecuta sin problemas
function suma (a,b) {
    return a+b;
}

y = inc(8);     // TypeError: inc is not a function
inc = function (x) {return ++x;};
```

NOTACIÓN LAMBDA (I)

Existe una sintaxis más sencilla para denotar funciones anónimas.

Las funciones escritas con esta notación se denominan **funciones flecha**

En lugar de:

```
function (x, y, z) { /* ... */ }
```

Puede escribirse:

```
(x, y, z) => { /* ... */ }
```

NOTACIÓN LAMBDA (II)

Si la función anónima solo tiene un parámetro pueden omitirse los paréntesis iniciales.

En lugar de:

```
function (x) { console.log(`Valor recibido: ${x}`); }
```

Puede escribirse:

```
x => { console.log(`Valor recibido: ${x}`); }
```

NOTACIÓN LAMBDA (III)

Además, si el cuerpo de la función es de la forma `return exp`, pueden omitirse las llaves y el `return`.

En lugar de:

```
function (x) { return x + 1; }
```

Puede escribirse:

```
x => x + 1
```

En el ejemplo anterior:

```
aplicar_funciones(  
  [ x => x - 3,  
    x => Math.sqrt(x),  
    factorial,  
    x => Math.log(x)  ], 2);
```

NUMERO DE ARGUMENTOS EN LA LLAMADA (I)

El número de argumentos en la llamada a la función no tiene que coincidir con el número de parámetros en la definición.

```
function imprimirArgumentos(a1,a2,a3) {  
  console.log(`a1: ${a1}`);  
  console.log(`a2: ${a2}`);  
  console.log(`a3: ${a3}`);  
}
```

```
imprimirArgumentos(1, true, "foo");
```

```
a1: 1  
a2: true  
a3: foo
```

NUMERO DE ARGUMENTOS EN LA LLAMADA (II)

- Si se proporcionan argumentos «de más» se ignoran los sobrantes:

```
imprimeArgumentos("uno", "dos", "tres", "cuatro");
```

```
a1: uno  
a2: dos  
a3: tres
```

- Si faltan argumentos, los parámetros correspondientes tomarán el valor **undefined**.

```
imprime_args("uno", "dos");
```

```
a1: uno  
a2: dos  
a3: undefined
```

PARÁMETROS POR DEFECTO

Desde ES6 es posible asignar valores por defecto a los parámetros de las funciones.

El valor por defecto se aplica si el parámetro no se pasa en la llamada o bien si se pasa el valor `undefined`.

```
function multiplicar(a, b = 1) {  
  return a*b;  
}
```

```
multiplicar(5); // 5  
multiplicar(5,undefined); // 5
```

PARÁMETROS NOMINALES

Utilizando *objetos* podemos *simular* el paso de *parámetros nominales*.

Por ejemplo, supongamos una función `abrir_fichero` que espera un nombre de fichero y, opcionalmente:

- Un parámetro `solo_lectura` que indica si el fichero se abre en modo lectura o en modo lectura/escritura.

Valor por defecto: `true`

- Un parámetro `binario` que indica si el fichero es binario o no.

Valor por defecto: `false`

EJEMPLOS DE LLAMADAS (I)

```
abrir_fichero("mio.txt");  
// Abriendo fichero mio.txt en modo lectura  
  
abrir_fichero("mio.txt", { solo_lectura: false });  
// Abriendo fichero mio.txt en modo lectura/escritura  
  
abrir_fichero("mio.txt", { binario: true });  
// Abriendo fichero binario mio.txt en modo lectura  
  
abrir_fichero("mio.txt", { binario: true, solo_lectura: false });  
// Abriendo fichero binario mio.txt en modo lectura/escritura  
  
abrir_fichero("mio.txt", { solo_lectura: true, binario: false });  
// Abriendo fichero mio.txt en modo lectura
```

EJEMPLOS DE LLAMADAS (II)

Implementación

```
// El objeto 'ops' tiene como atributos los parámetros nominales.  
  
function abrir_fichero(nombre, ops = {}) {  
    // Inicialización de los parámetros nominales no pasados  
    if (ops.solo_lectura === undefined) ops.solo_lectura = true;  
    if (ops.binario === undefined) ops.binario = false;  
  
    // Cuerpo de la función  
    console.log(`Abriendo fichero ${ops.binario ? "binario " : ""}` +  
        `${nombre} en modo ` +  
        `${ops.solo_lectura ? "lectura" : "lectura/escritura"}`);  
}
```

Sintaxis alternativa

```
function abrir_fichero(nombre, ops={solo_lectura:true, binario:false}  
  
    console.log(`Abriendo fichero ${ops.binario ? "binario " : ""}` +  
        `${nombre} en modo ` +  
        `${ops.solo_lectura ? "lectura" : "lectura/escritura"}`);  
}
```

EL OBJETO arguments

Es una variable local de las funciones (no flecha).

- Es un objeto "parecido" al objeto **Array**.
- Contiene los argumentos de la llamada a la función.
- Se puede acceder al número de argumentos de la llamada con su propiedad **length**.

```
function imprimeArgumentos(a,b,c) {  
    for (let i=0; i<arguments.length; i++) {  
        console.log(arguments[i]);  
    }  
}  
imprimeArgumentos(1,2,3); // Imprime 1 2 3
```

EJEMPLO DEL OBJETO arguments

```
//  
// Función que busca el mínimo de un conjunto de valores  
//  
function minimo() {  
    let min = arguments[0];  
    for (let i=0; i<arguments.length; i++) {  
        if (arguments[i]<min) {  
            min = arguments[i];  
        }  
    }  
    return min;  
}  
  
console.log(minimo()); // undefined  
console.log(minimo(1)); // 1  
console.log(minimo(3,4,5)); // 3  
console.log(minimo(9,8,7,6,5,4,3,2,1,0)); // 0
```

FUNCIONES DENTRO DE OBJETOS (I)

Las funciones pueden ser asignadas a los atributos de un objeto.

```
var empleado = {  
  nombre: "Manuel",  
  saludar: function() {  
    console.log("¡Hola!");  
  }  
};  
  
empleado.saludar();    // → ¡Hola!
```

¡No utilizar expresiones lambda aquí!

Este tipo de funciones reciben el nombre de **métodos**.

Se puede añadir métodos a un objeto ya construido.

```
empleado.despedir = function() { console.log("¡Adios!"); };  
empleado.despedir();    // → ¡Adios!
```

FUNCIONES DENTRO DE OBJETOS (II)

this

```
empleado.saludar();
```

En toda llamada a un método se distinguen tres componentes:

- Método llamado: **saludar**
- Argumentos (ninguno, en este caso)
- Objeto sobre el que se realiza la llamada: **empleado**

Cuando se llama a un método, éste recibe, además de los correspondientes argumentos, una variable especial (**this**) que contiene una **referencia al objeto sobre el que se realiza la llamada.**

FUNCIONES DENTRO DE OBJETOS (III)

Ejemplo

```
var empleado = {  
  nombre: "Manuel",  
  
  saludar: function() {  
    console.log(`¡Hola, ${this.nombre}!`);  
  },  
  
  cambiarNombre: function(nuevoNombre) {  
    this.nombre = nuevoNombre;  
  }  
};
```

```
empleado.saludar();    // → ¡Hola, Manuel!  
  
empleado.cambiarNombre("Irene");  
  
empleado.saludar();    // → ¡Hola, Irene!
```

FUNCIONES DENTRO DE OBJETOS (IV)

Pueden transferirse métodos entre distintos objetos.

```
var otro_empleado = {  
  nombre: "David",  
  saludar: empleado.saludar  
};  
  
otro_empleado.saludar(); // → ¡Hola, David!
```

Se imprime el nombre de `otro_empleado`, porque es el objeto que recibe la llamada, aunque se llame a un método proveniente de otro objeto.

CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



INICIALIZACIÓN DE ARRAYS

Un array puede inicializarse enumerando sus elementos:

```
let a = [23, 12, 69, 11, 34, 45];
```

o bien mediante el constructor **Array**:

```
let b = new Array(10);  
// Todos los elementos tienen el valor 'undefined'
```

LOS ARRAYS SON OBJETOS

Es posible asignar propiedades arbitrarias a un array.

```
let a = [23, 12, 69, 11, 34, 45];  
a.estaOrdenado = false;  
  
console.log(a);  
    // → [ 23, 12, 69, 11, 34, 45, esta_ordenado: false ]
```

Todos los arrays extienden a la clase **Array**, que contiene algunos métodos de utilidad sobre arrays. [\[+\]](#)

LOS ARRAYS SON FLEXIBLES (I)

LOS ARRAYS SON FLEXIBLES Y PUEDEN TENER «HUECOS»

Puede variarse la longitud de un array en tiempo de ejecución. Basta con modificar la propiedad `length`:

```
let a = [23, 12, 69, 11, 34, 45];  
a.length += 2; // Ampliamos el array  
  
console.log(a); // → [ 23, 12, 69, 11, 34, 45, ,  ]  
  
a.length = 3; // Reducimos el array  
  
console.log(a); // → [ 23, 12, 69 ]
```

También se puede ampliar el array añadiendo elementos fuera de su rango:

```
a[5] = 32;  
console.log(a); // → [ 23, 12, 69, , , 32 ]
```

LOS ARRAYS SON FLEXIBLES (II)

Métodos que modifican el tamaño del array:

- `push(x)`
Inserta `x` al final del array.
- `pop()`
Elimina y devuelve el último elemento del array.
- `unshift(x)`
Añade `x` al principio del array, desplazando los restantes elementos.
- `shift()`
Elimina el primer elemento del array, desplazando los restantes elementos.
- `splice(ini, num)` Partiendo del elemento en la posición `ini`, elimina `num` elementos.

OPERACIONES DESTRUCTIVAS

```
let a = [1, 2, 3, 4, 5];  
    // a = [1, 2, 3, 4, 5];  
  
a.push(8);  
    // a = [1, 2, 3, 4, 5, 8];  
  
a.unshift(-4);  
    // a = [-4, 1, 2, 3, 4, 5, 8];  
  
a.pop(); // → 8  
    // a = [-4, 1, 2, 3, 4, 5];  
  
a.shift(); // → -4  
    // a = [1, 2, 3, 4, 5];  
  
a.splice(2, 2); // → [3, 4]  
    // a = [1, 2, 5];
```

```
a = [4, 7, 4, 1, 3, 5];  
a.sort();  
    // a = [1, 3, 4, 4, 5, 7]  
  
a.reverse();  
    // a = [7, 5, 4, 4, 3, 1]
```

OPERACIONES NO DESTRUCTIVAS

- `concat(arr_1, ..., arr_n)`

Añade los arrays pasados como argumento y devuelve el resultado.

```
[1, 2, 3].concat([4, 5], [6, 7, 8]);  
// → [1, 2, 3, 4, 5, 6, 7, 8]
```

- `slice(ini, fin)`

Devuelve el segmento `[ini, fin)` del array.

```
["a", "b", "c", "d", "e", "f", "g"].slice(2, 5);  
// → ["c", "d", "e"]
```

- `join(sep)`

Concatena los elementos del array intercalando `sep` como separador:

```
["Esto", "no", "me", "gusta"].join(" - ");  
// → "Esto - no - me - gusta"
```

BÚSQUEDA DE VALORES

- `indexOf(elem, [pos_inicial])`

Devuelve el índice de la primera aparición de `elem` en el array a partir de `pos_inicial`, o -1 si no se encuentra.

- `lastIndexOf(elem)`

Devuelve el índice de la última aparición de `elem` en el array, o -1 si no se encuentra.

FUNCIONES DE ORDEN SUPERIOR

Una función de **orden superior** es una función que recibe funciones como parámetro y/o devuelve funciones.

Javascript proporciona varios métodos de orden superior para arrays que son muy útiles en la práctica.

FUNCIONES DE ITERACIÓN (I)

- `forEach(f)`

Aplica la función `f` sobre todos los elementos del array.

```
let personas = [ { nombre: "Ricardo", edad: 45},  
                  { nombre: "Julia", edad: 24 },  
                  { nombre: "Ashley", edad: 28 } ];  
  
personas.forEach(p => {  
    console.log("Hola, me llamo " + p.nombre  
                + " y tengo " + p.edad + " años");  
})
```

La salida esperada es:

```
Hola, me llamo Ricardo y tengo 45 años  
Hola, me llamo Julia y tengo 24 años  
Hola, me llamo Ashley y tengo 28 años
```

FUNCIONES DE ITERACIÓN (II)

En la función `forEach(f)`, `f` tiene tres parámetros.

- `v`: el valor de elemento en proceso de array.
- `i`: el índice del elemento en proceso dentro del array .
- `a`: el array total.

```
personas.forEach((v,i,a) => {  
    console.log("Hola,me llamo " + v.nombre  
        + " y tengo " + v.edad + " años"  
        + " y soy el " + i + " de un array de "  
        + a.length +" elementos" );  
})
```

La salida esperada es:

```
Hola,me llamo Ricardo y tengo 45 años, soy el 0 de un array de 3 elemen  
Hola,me llamo Julia y tengo 24 años, soy el 1 de un array de 3 elemer  
Hola,me llamo Ashley y tengo 28 años, soy el 2 de un array de 3 eleme
```

FUNCIONES DE TRANSFORMACIÓN

- `map(f) → f(v,i,a)`

Aplica la función `f` a cada elemento del array, devolviendo otro array con los resultados.

```
var a = [1, 3, 5, 2, 4];  
let dobles = a.map(n => n * 2);  
console.log(dobles);    // → [2, 6, 10, 4, 8]
```

- `filter(f) → f(v,i,a)`

Selecciona los elementos `x` del array tales que `f(x)` devuelve `true` y devuelve un array con dichos elementos.

```
var a = [1, 3, 5, 2, 4];  
let pares = a.filter(n => n % 2 === 0);  
console.log(pares);    // → [2, 4]
```

FUNCIONES DE REDUCCIÓN (I)

- $\text{every}(f) \rightarrow f(v, i, a)$

Devuelve **true** si para *todo* elemento **x** del array, **f(x)** devuelve **true**.

```
var a = [1, 3, 5, 2, 4];
var sonNumeros = a.every( n => typeof n === "number");
console.log(sonNumeros);    // → true

var b=[1, 2, "foo"];
var sonNumeros = b.every( n => typeof n === "number");
console.log(sonNumeros);    // → false
```

- $\text{some}(f) \rightarrow f(v, i, a)$

Devuelve **true** si *existe* un elemento **x** en el array tal que **f(x)** devuelva **true**.

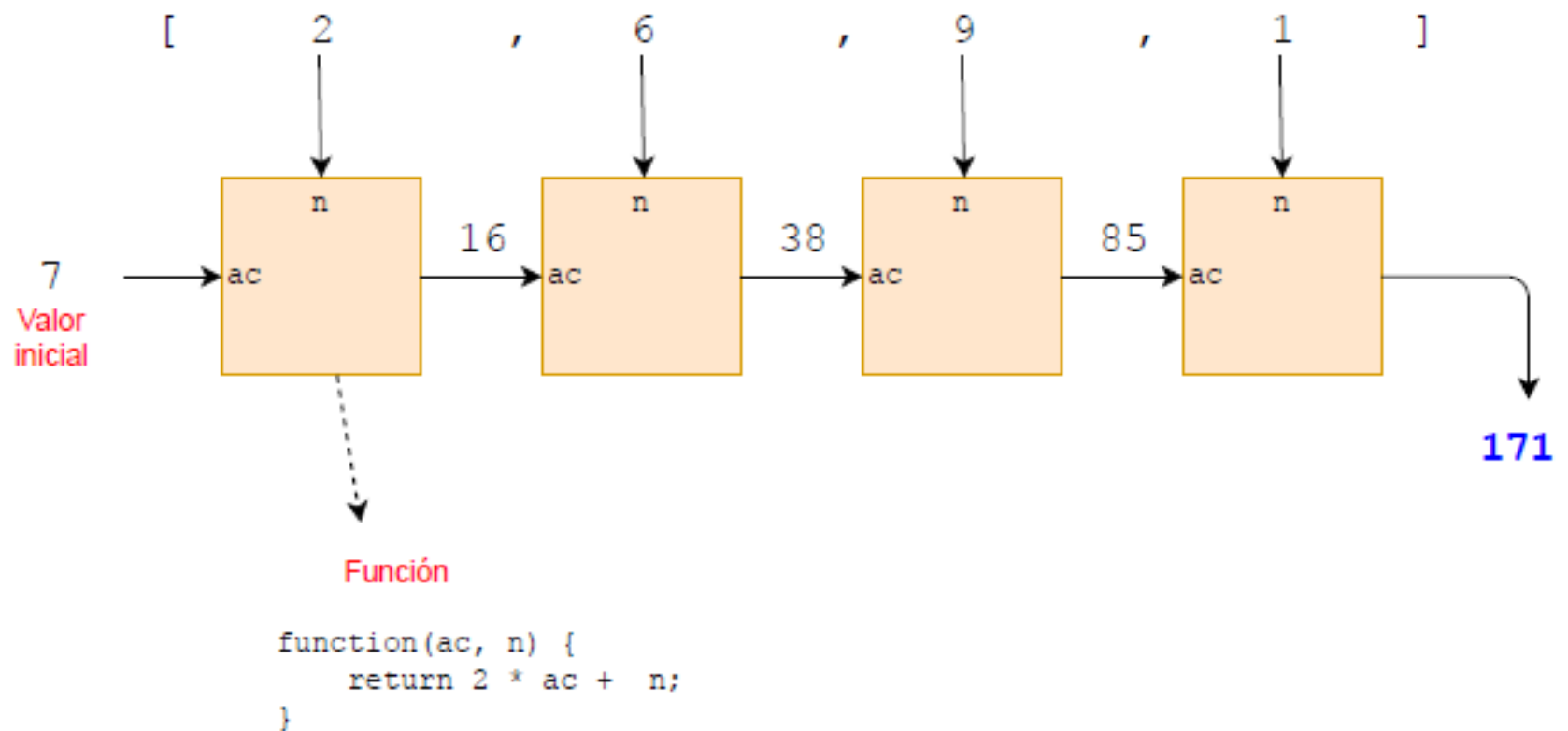
FUNCIONES DE REDUCCIÓN (II)

- `reduce(f, [valorInicial])` $\rightarrow f(ac, v, i, a)$

Recorre el array de izquierda a derecha, acumulando un valor durante el recorrido.

```
let a = [2, 6, 9, 1];  
  
console.log(  
  "Valor final: " +  
  a.reduce((ac, n) => 2 * ac + n, 7)); // → 171
```

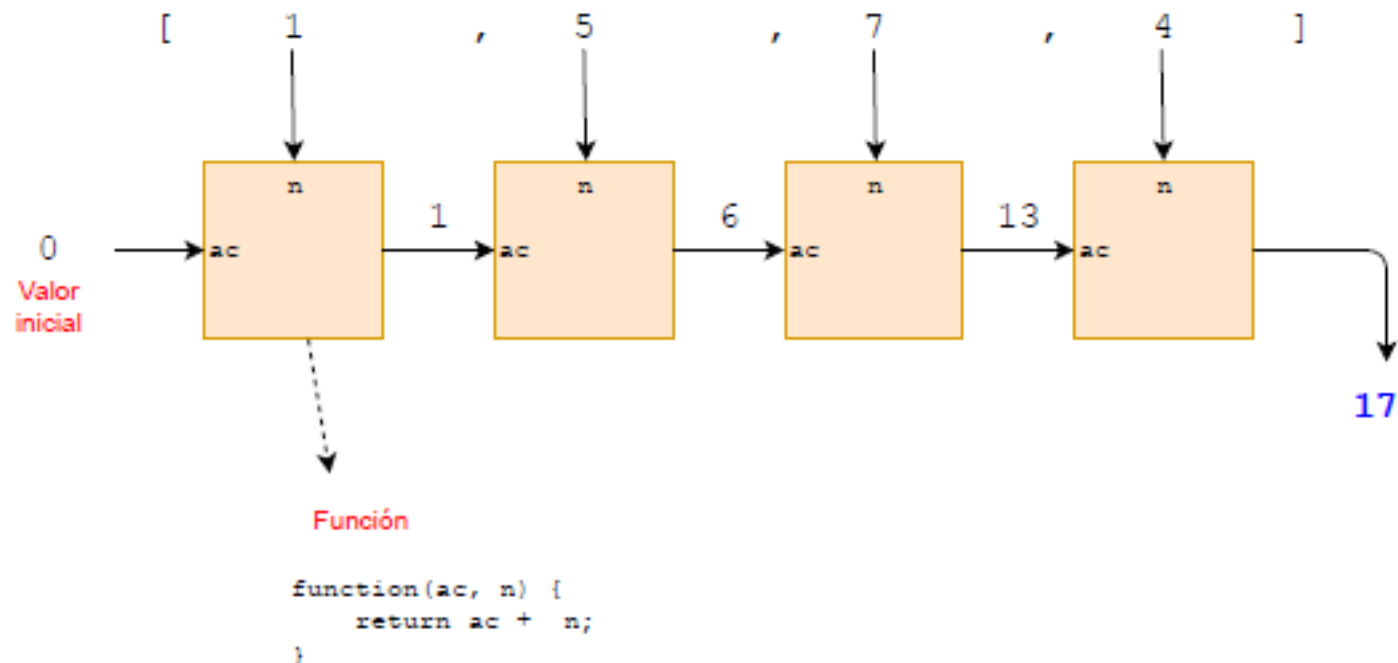
FUNCIONES DE REDUCCIÓN (III)



FUNCIONES DE REDUCCIÓN (IV)

Suma de los elementos de un array

```
[1, 5, 7, 4].reduce((ac, n) => ac + n, 0)
```



FUNCIONES DE REDUCCIÓN (V)

Multiplicación de los elementos de un array

```
[1, 5, 7, 4].reduce((ac, n) => ac * n, 1)
```

Máximo de los elementos de un array

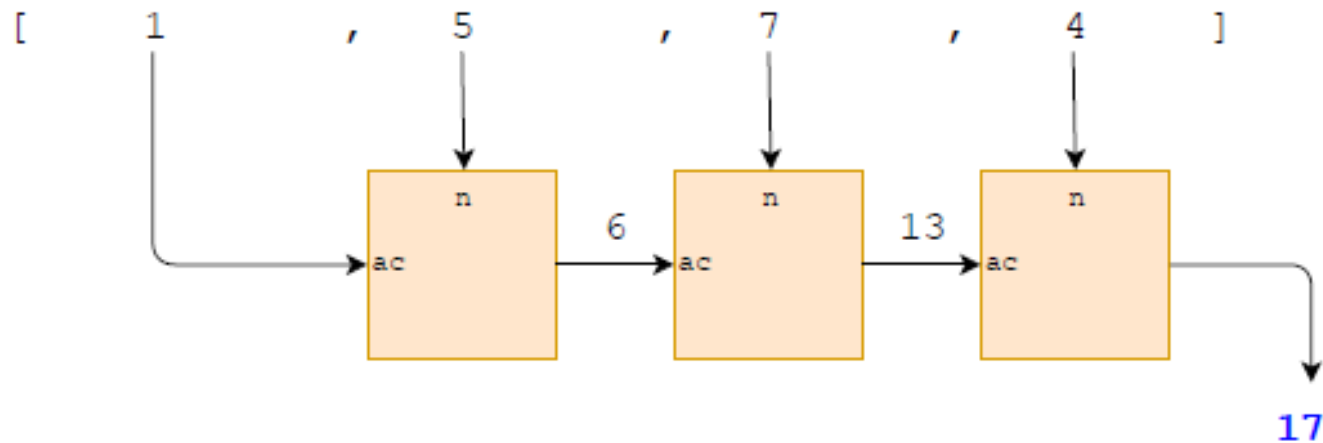
```
[6, 1, 4, 3, 7].reduce((ac, x) => Math.max(ac, x), -Infinity);
```

FUNCIONES DE REDUCCIÓN (VI)

`reduce(f)`

Si no se indica valor inicial, se supone que éste es el primer elemento del array.

```
[1, 5, 7, 4].reduce((ac, x) => ac + x)
```



FUNCIONES DE REDUCCIÓN (VII)

VARIANTES

`reduceRight(f, [valorInicial])`

Realiza lo mismo que `reduce`, pero recorriendo el vector de derecha a izquierda.

CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



DEFINICIÓN

Una expresión regular es un **patrón** que representa una o varias cadenas de texto.

Se utilizan para operaciones de búsqueda y sustitución de texto.

Por ejemplo, la expresión regular **[A-Z][0-9]{3}** denota el conjunto de cadenas que comienzan por una letra mayúscula y van seguidas por tres dígitos

Ejs: **A324**, **F983**, etc.

Ver: [Lenguaje de expresiones regulares](#)

ESCRITURA DE EXPRESIONES REGULARES

Una expresión regular puede contener dos tipos de caracteres:

- **Caracteres simples:** se representan a sí mismos.

Ej: la expresión regular */abc/* representa la cadena *abc*.

- **Metacaracteres:** tienen un significado especial.

Ej: la expresión regular */a+/* representa las cadenas *a*, *aa*, *aaa*, *aaaa*, etc.

El símbolo *+* es un metacarater que significa *una o más ocurrencias*.

ALGUNOS METACARACTERES (I)

-
- + Una o más ocurrencias de la expresión que lo precede.

Ej: $/a+/ \rightarrow a, aa, aaa, \dots$

Concuerda con a en "café", con todas las a es en "holaaaaaa" y con nada en "te".

-
- * Cero o más ocurrencias de la expresión que lo precede.

Ej: $/ab*/ \rightarrow a, ab, abbbbb, \dots$

Concuerda con a en "café", con ab en "cable" y con nada en "lejos".

ALGUNOS METACARACTERES (II)

?

Cero o una ocurrencias de la expresión que lo precede.

Ej: `/a?bc?/` → *b, ab, bc, abc*

Concuerda con *b* en "bar", con *abc* en "el diario abc" y con nada en "sol".

.

Cualquier caracter excepto salto de línea.

Ej: `.*` → una línea entera.

{n}

n ocurrencias de la expresión que lo precede.

Ej: `/a{2}/` → *ee*

Concuerda con *ee* en "releer" y con las dos primeras ocurrencias de *e* en "beeeeeeeee".

ALGUNOS METACARACTERES (III)

**** Si precede a un metacaracter hace que pierda su significado especial.

Ej: **/*/** → representa el caracter "*"

Ej: **/\./** → representa el caracter "."

Ej: **/** → representa el caracter "\"

Si precede a ciertos caracteres simples, los transforma en caracteres especiales.

Ej: **/\n/** → salto de línea

Ej: **/\t/** → tabulador

Ej: **/\s/** → espacio en blanco (incluye también tabulador, salto de línea, etc)

ALGUNOS METACARACTERES (IV)

[xyz] Uno de los elementos de los indicados entre corchetes. Con el símbolo "-" se expresan rangos.

Ej: **/[abc]/** → *a*, *b* o *c*

Ej: **/[a-zA-Z]/** → cualquier letra minúscula o mayúscula

\w Caracter alfanumérico o guión bajo. Es equivalente a **[a-zA-Z0-9_]**.

Ej: **/\w/** → Concuerda con *a* en "ab", con 5 en "5\$" y con _ en "%_%_"

Ej: **/[a-zA-Z]/** → cualquier letra minúscula o mayúscula

\d Caracter numérico. Es equivalente a **[0-9]**.

CREACIÓN DE EXPRESIONES REGULARES

En JavaScript las expresiones regulares se pueden crear de dos maneras:

- De manera literal: patrón delimitado por el símbolo /

```
var expr = /[A-Z][0-9]{3}/;
```

Este método es adecuado cuando la expresión regular es constante.

- Utilizando la clase **RegExp**

```
var expr = new RegExp('[A-Z][0-9]{3}');
```

Este método es adecuado cuando la expresión regular no es constante, por ejemplo, cuando proviene de un usuario.

MÉTODOS DE LA CLASE RegExp (I)

- `test(str)`: devuelve `true` si en la cadena `str` contiene una subcadena que encaja con la expresión, o `false` en caso contrario.

```
[A-Z][0-9]{3}/.test("A655");    // → true
[A-Z][0-9]{3}/.test("__A655__");  // → true
[A-Z][0-9]{3}/.test("Otra cosa"); // → false

let idValido = /\b[a-zA-Z_][a-zA-Z0-9_]*\b/;
idValido.test("8cte");    // → false
idValido.test("_cte");    // → true
```

`\b` significa separador de palabra.

MÉTODOS DE LA CLASE RegExp (II)

- `exec(str)`: busca en la cadena una subcadena que concuerde con la expresión y devuelve el texto encontrado, o `null` en caso contrario.

```
/\d{3}/.exec("123 456 789"); // → ["123", index: 0, ...]  
  
/\w+@\w+/.exec("abc"); // → null  
  
/\w+@\w+/.exec("Esto es un correo nombre_apellidos@correo");  
// → ["nombre_apellido@correo", index: 18,...]
```

MÉTODOS DE LA CLASE RegExp (III)

A veces se puede dividir la expresión regular en varios grupos con el fin de saber qué parte de la cadena capturada corresponde a cada grupo.

Cada grupo va delimitado entre paréntesis (,)

La función `exec` nos permite desglosar cualquier cadena que ajuste con el patrón en sus distintos grupos.

MÉTODOS DE LA CLASE RegExp (IV)

Por ejemplo, la siguiente expresión:

`\d{4}\-[A-Z]{3}`

ajusta con cualquier secuencia de cuatro dígitos (`\d` = dígito) que vaya seguida de un guión (`\-`) y tres letras mayúsculas.

Ejs: `0249-GSW`, `1934-HHG`, etc.

Si queremos poder separar la secuencia de dígitos de la de letras utilizamos dos grupos:

`(\d{4})\-([A-Z]{3})`

MÉTODOS DE LA CLASE RegExp (V)

```
var regexp = /(\d{4})\-([A-Z]{3})/;  
var result = regexp.exec("Mi matrícula de coche es 8367-AWD");
```

La subcadena **8367**-**AWD** ajusta con el patrón **regexp**, pero **exec** nos permite saber qué fragmento de ésta ajusta con cada grupo

```
result[0] // → "8367-AWD" (Cadena completa)  
result[1] // → "8367"      (Primer grupo de captura)  
result[2] // → "AWD"       (Segundo grupo de captura)  
  
result.index // → 25      (Posición del ajuste dentro de la cadena)
```


MODIFICADORES DE EXPRESIONES REGULARES

Se colocan tras el delimitador `/` final de la expresión.

<code>i</code>	No distingue entre mayúsculas y minúsculas.
<code>g</code>	Ajuste global. Permite encontrar varias ocurrencias en la misma cadena.
<code>m</code>	Buscar a lo largo de varias líneas.

EXPRESIONES REGULARES Y MÉTODOS DE CADENAS

Es habitual el uso de expresiones regulares con métodos de cadenas para realizar operaciones de búsqueda y sustitución de texto.

- `match(regex)`: devuelve todas las subcadenas que ajustan con la expresión regular `regex` (si ésta contiene el modificador `g`) o solamente la primera (en caso contrario).

```
var str = "a aa aaa";
str.match(/a+/); // → ["a", index: 0, input:"a aa aaa",groups:undefi
str.match(/a+/g); // → ["a", "aa", "aaa"]
str = "HolA";
str.match(/hola/); // → null
str.match(/hola/i); // → ["HolA",index:0, input:"HolA", groups:undefi
```

EXPRESIONES REGULARES Y MÉTODOS DE CADENAS

- `search(regex)`: devuelve el índice de la primera subcadena que ajuste con `regex`, o -1 si no hay ninguna.

```
var str = "123 456 789";  
str.search(/\d{3}/);    // → 0  
str.search(/3\d{2}/);    // → -1
```

- `replace(regex, nuevaCadena)`: reemplaza por `nuevaCadena` las subcadenas que ajusten con `regex`.

```
var str = "123 456 789";  
str.replace(/\d{3}/, "AAA");    // → AAA 456 789  
str.replace(/\d{3}/g, "AAA");    // → AAA AAA AAA
```

CONTENIDO

- INTRODUCCIÓN
- CONCEPTOS BÁSICOS
- LOS TIPOS DEL LENGUAJE
- OBJETOS
- FUNCIONES
- ARRAYS
- EXPRESIONES REGULARES
- ORIENTACIÓN A OBJETOS BASADA EN PROTOTIPOS
- BIBLIOGRAFÍA



LENGUAJE OO BASADO EN CLASES VS. PROTOTIPOS

Hay dos tipos lenguajes orientados a objetos:

- Lenguajes OO **basados en clases** (Java).
- Lenguajes OO **basados en prototipos** (JavaScript).

Ambos modelos difieren en aspectos muy importantes:

- Elementos presentes en el modelo.
- Mecanismo de creación de objetos.
- Herencia.
- Ampliación de las propiedades de los objetos
- ...

ELEMENTOS PRESENTES EN EL MODELO

OO-clases

Clases y
objetos (instancias).

OO-prototipos

Sólo objetos.
Todos tienen la misma
categoría.
Algunos objetos son **prototipos**
de otros.

MECANISMOS DE CREACIÓN DE OBJETOS

OO-clases

OO-prototipos

Constructores.

Creación literal.

Creación a partir de otro
objeto (prototipo).

CREACIÓN LITERAL (I)

Se utiliza el mecanismo que se muestra en la figura.

```
let c = {  
  r:1,  
  i:1,  
  modulo: function() {  
    return Math.sqrt(this.r*this.r +  
                      this.i*this.i);  
  },  
  argumento: function() {  
    return Math.atan2(this.i,this.r);  
  }  
};
```

Se crea el objeto **c** que contiene **sus** atributos(**r** e **i**) y **sus** métodos (**modulo** y **argumento**).

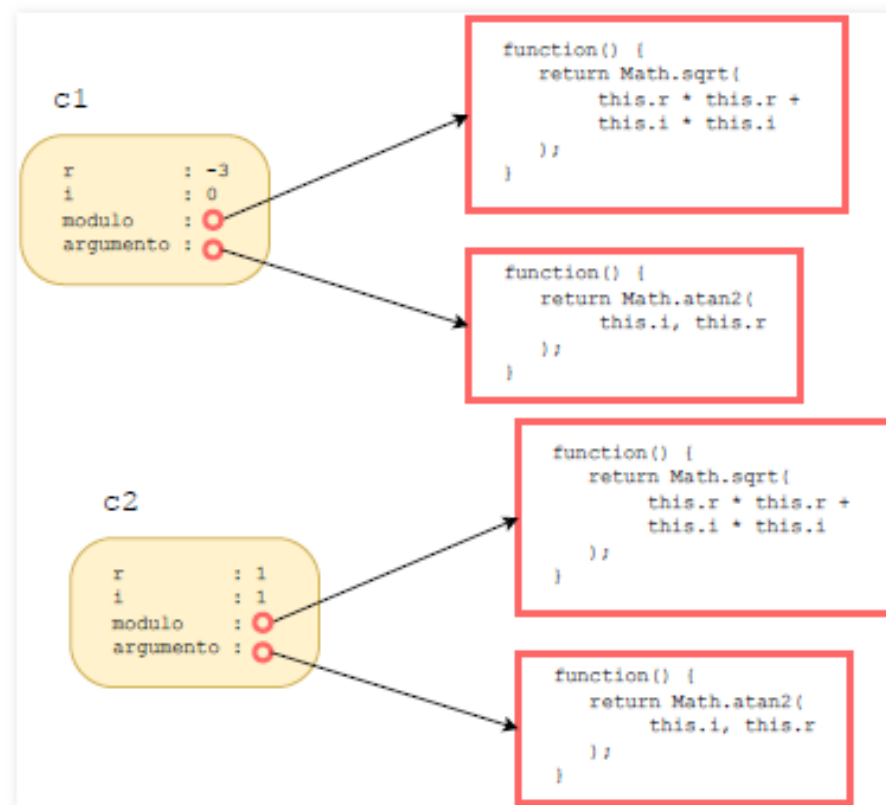
CREACIÓN LITERAL (II)

Se puede encapsular la creación de objetos mediante funciones constructoras.

```
function construirComplejo(real, imag) {  
  return {  
    r : real,  
    i : imag,  
  
    modulo: function() {  
      return Math.sqrt(this.r * this.r + this.i * this.i);  
    },  
  
    argumento: function() {  
      return Math.atan2(this.i, this.r);  
    }  
  }  
}  
  
var c1 = construirComplejo(-3, 0);  
console.log(c1.argumento()); // → 3.141592653589793  
var c2 = construirComplejo(1, 1);  
console.log(c2.modulo());    // → 1.4142135623730951
```

PROBLEMA

Duplicidad de objetos función para cada objeto.



¿No podrían c1 y c2 compartir los métodos?

POSIBLE SOLUCIÓN (I)

```
function moduloComplejo() {  
    return Math.sqrt(this.r * this.r + this.i * this.i);  
}  
  
function argumentoComplejo() {  
    return Math.atan2(this.i, this.r);  
}  
  
function construirComplejo(real, imag) {  
    return {  
        r : real,  
        i : imag,  
        modulo: moduloComplejo,  
        argumento: argumentoComplejo  
    }  
}
```

POSIBLE SOLUCIÓN (II)

c1

r : -3
i : 0
modulo : 
argumento : 

c2

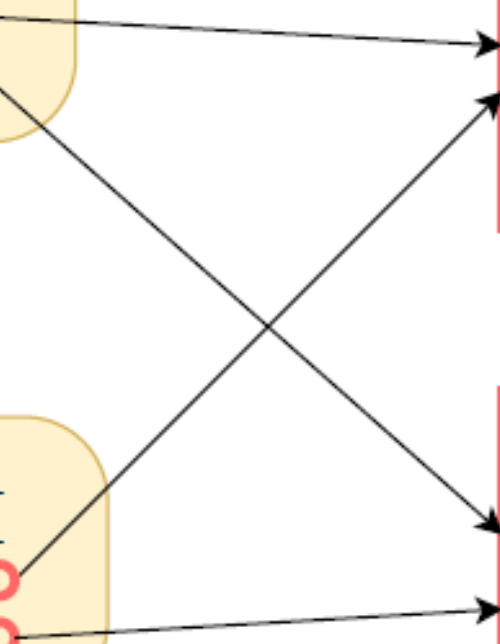
r : 1
i : 1
modulo : 
argumento : 

moduloComplejo

```
function() {  
    return Math.sqrt(  
        this.r * this.r +  
        this.i * this.i  
    );  
}
```

argumentoComplejo

```
function() {  
    return Math.atan2(  
        this.i, this.r  
    );  
}
```



POSIBLE SOLUCIÓN (III)

Si se añade un método nuevo a **c1**.

```
var c1 = construirComplejo(-3, 0);  
var c2 = construirComplejo(1, 1);  
  
// ...  
  
c1.coordenadasPolares = function() {  
    console.log(`(${this.modulo()}), ${this.argumento()} `);  
}
```

Este método existe solamente dentro de **c1**.

¿Existe alguna manera de añadir un método simultáneamente a todos los objetos que hubiesen sido creados mediante **construirComplejo**?

Sí. Se puede hacer mediante **prototipos**.

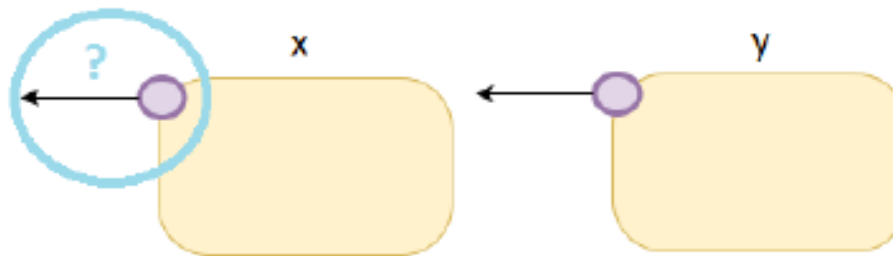
CREACIÓN A PARTIR DE OTRO OBJETO (PROTOTIPO)

Se utiliza la función `Object.create()`

La siguiente sentencia:

```
let y = Object.create(x);
```

crea un objeto `y` que tiene a `x` como prototipo.



El objeto `x` tendrá su prototipo.

EJEMPLO (I)

Creación del objeto **c1** a partir del objeto **c**.

```
let c = {  
  r:1,  
  i:1,  
  modulo: function() {  
    return Math.sqrt(this.r*this.r  
                      + this.i*this.i  
  },  
  argumento: function() {  
    return Math.atan2(this.i, this.r)  
  }  
};  
  
let c1 = Object.create(c);
```

Visualización de los objetos en la consola.

The screenshot shows a web browser's developer console with the 'Console' tab selected. The console displays the following sequence of commands and results:

- `> c`
Result: `{r: 1, i: 1, modulo: f, argumento: f}`
- `> c1`
Result: `{}`
- `> c1.r`
Result: `1`
- `> c1.i`
Result: `1`
- `> c1.modulo()`
Result: `1.4142135623730951`
- `> c1.argumento()`
Result: `0.7853981633974483`

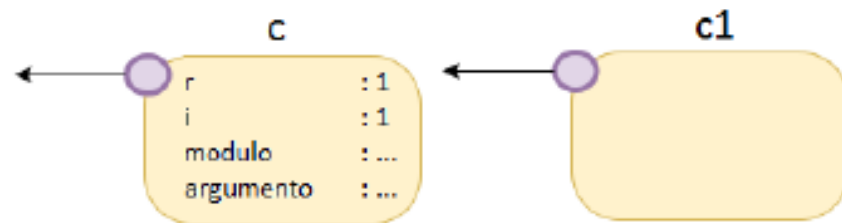
Annotations on the right side of the console output:

- A red vertical line is placed next to the `c1` result, with the text **SIN ATRIBUTOS** to its right.
- A blue vertical line is placed next to the `c1.r` and `c1.i` results, with the text **VALORES** to its right.
- A blue vertical line is placed next to the `c1.modulo()` and `c1.argumento()` results, with the text **ATRIBUTOS** to its right.

EJEMPLO (II)

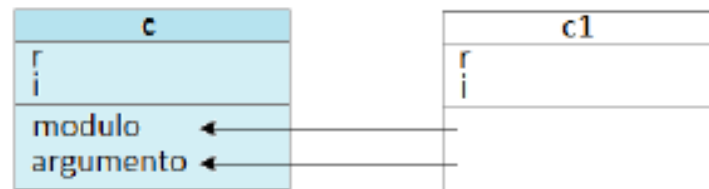
Creación de un objeto en JavaScript con `Object.create()`.

```
let c1= Object.create(c)
```



Creación de un objeto en un lenguaje basado en clases.

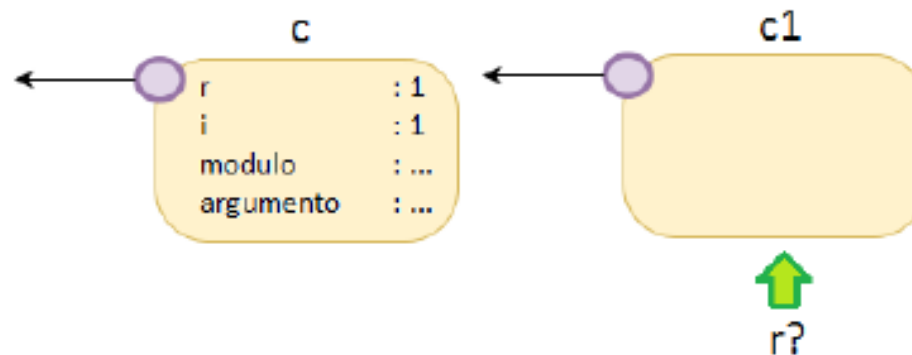
```
class c {  
    r ...  
    i ...  
    modulo ...  
    argumento ...  
}  
c1 = new c(...)
```



BÚSQUEDA DE ATRIBUTOS (I)

¿Qué ocurre cuando se accede al atributo **r** de **c1**?

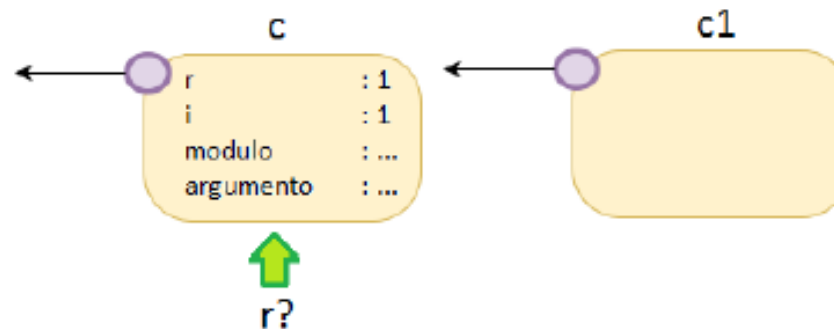
En primer lugar se intenta buscar **r** dentro de **c1**.



r no se encuentra en **c1** porque como se ha visto en la consola, el objeto **c1** no tiene atributos. Se sigue buscando...

BÚSQUEDA DE ATRIBUTOS (II)

... dentro del prototipo de **c1**, que es **c**.



El atributo **r** se encuentra en **c** y se detiene la búsqueda.

Si no se hubiera encontrado, la búsqueda continuaría por la **cadena de prototipos** hasta que:

- Se encuentre el atributo en algún objeto de la cadena.
- Se llegue al final de la cadena. En ese caso la expresión **c1.r** se evaluaría a **undefined**.

BÚSQUEDA DE ATRIBUTOS (III)

- Cada objeto mantiene un enlace a otro objeto que es su **prototipo**.
- Ese objeto prototipo tiene su propio prototipo, y así sucesivamente formando una **cadena de prototipos**.
- Hasta que se alcanza un objeto cuyo prototipo es **null**, y actúa como raíz de la cadena de prototipos.



- El acceso a un atributo de un objeto se hace partiendo del propio objeto y ascendiendo por la cadena de prototipos.

BÚSQUEDA DE ATRIBUTOS (IV)

En el ejemplo previo, **c1** no tiene **atributos propios**.

Para que un objeto tenga como propio un atributo existente en su cadena de prototipos, hay que crear dicho atributo asignándole un valor. Este valor "apantalla" al presente en la cadena de prototipos.

```
c1.i = 8;  
c1.r = 0;
```

The screenshot shows a browser's developer console with the following interactions:

- Command: `> c`
Result: `< {r: 1, i: 1, modulo: f, argumento: f}`
- Command: `> c1.r = 8;`
Result: `< 8`
- Command: `> c1.i = 0;`
Result: `< 0`
- Command: `> c1`
Result: `< {r: 8, i: 0}`

Annotations on the right side of the console:

- A yellow vertical bar highlights the first two commands, with the text **ASIGNACIÓN DE VALORES** next to it.
- A green vertical bar highlights the last command, with the text **ATRIBUTOS "PROPIOS"** next to it.

AÑADIR ATRIBUTOS AL PROTOTIPO (I)

AÑADIR ATRIBUTOS AL PROTOTIPO

Se define el objeto `circulo` y se usa como prototipo para la creación de los objetos `c1` y `c2`.

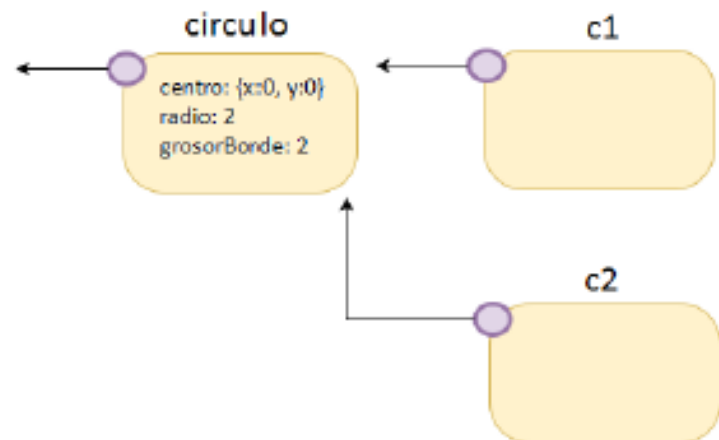
```
let circulo = {  
  centro: { x: 0, y: 0 },  
  radio: 5  
};  
  
let c1 = Object.create(circulo);  
let c2 = Object.create(circulo);
```

Si se añaden nuevos atributos a `circulo`, los objetos `c1` y `c2` los heredan automáticamente.

```
circulo.grosorBorde = 2;  
  
console.log(c1.grosorBorde); // → 2  
console.log(c2.grosorBorde); // → 2
```

AÑADIR ATRIBUTOS AL PROTOTIPO (II)

```
let circulo = {  
  centro: {x: 0, y: 0},  
  radio: 5  
};  
  
let c1 = Object.create(circulo);  
let c2 = Object.create(circulo);  
  
circulo.grosorBorde = 2;  
  
console.log(c1.grosorBorde); // → 2  
console.log(c1.grosorBorde); // → 2  
  
console.log(c1.radio); // → 5  
console.log(c2.centro); // → {x:0, y:0}
```

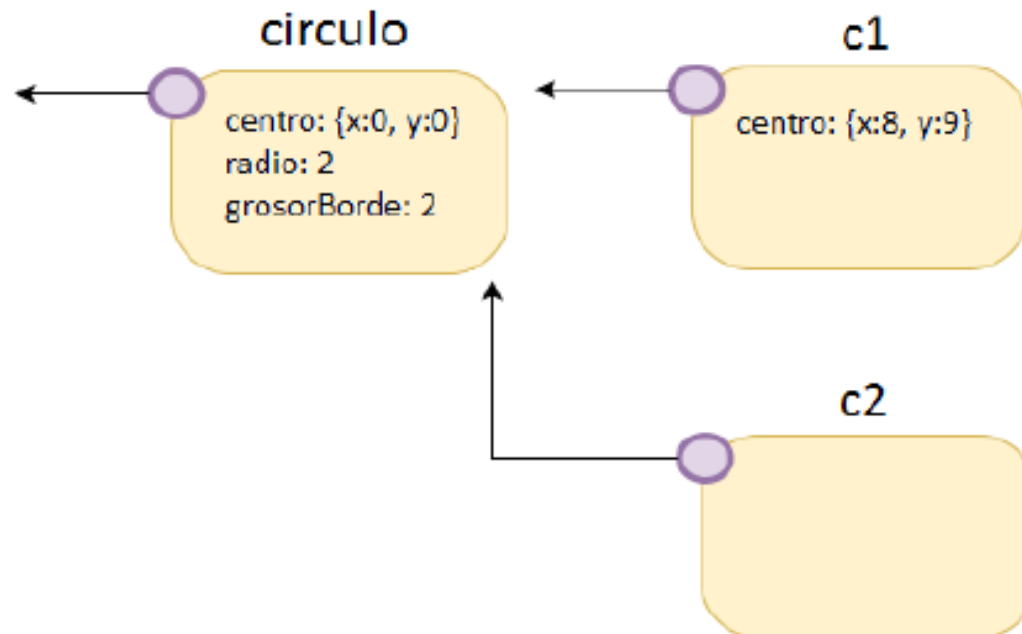


Los objetos **c1** y **c2** no tienen atributos propios.
Cualquier acceso a los atributos desencadena la búsqueda
en la cadena de prototipos.

AÑADIR ATRIBUTOS AL PROTOTIPO (III)

En el objeto **c1** se asigna valor al atributo **centro**.

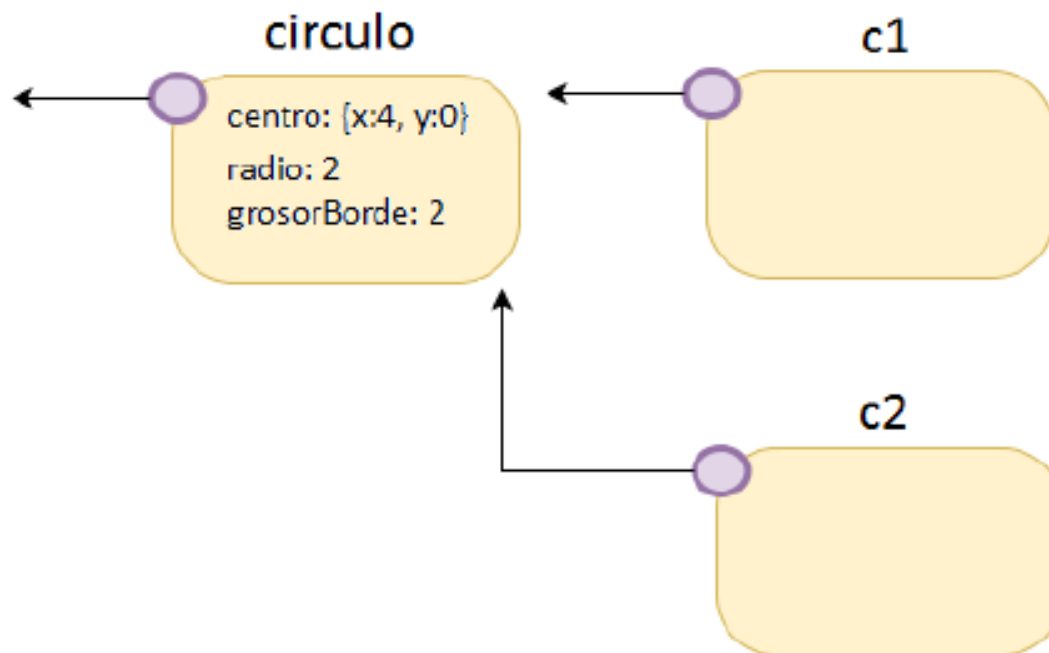
```
c1.centro = {x:8, y:9};  
  
console.log(c1.centro); // → {x:8, y:9}  
console.log(c2.centro); // → {x:0, y:0}
```



AÑADIR ATRIBUTOS AL PROTOTIPO (IV)

En el objeto **c1** se asigna valor al atributo **centro.x**.

```
c1.centro.x = 4;  
  
console.log(circulo.centro); // → {x:4, y:0}  
console.log(c1.centro); // → {x:4, y:0}  
console.log(c2.centro); // → {x:4, y:0}
```



EL OBJETO Object.prototype

En la raíz de la cadena de prototipos se encuentra el objeto `Object.prototype`.

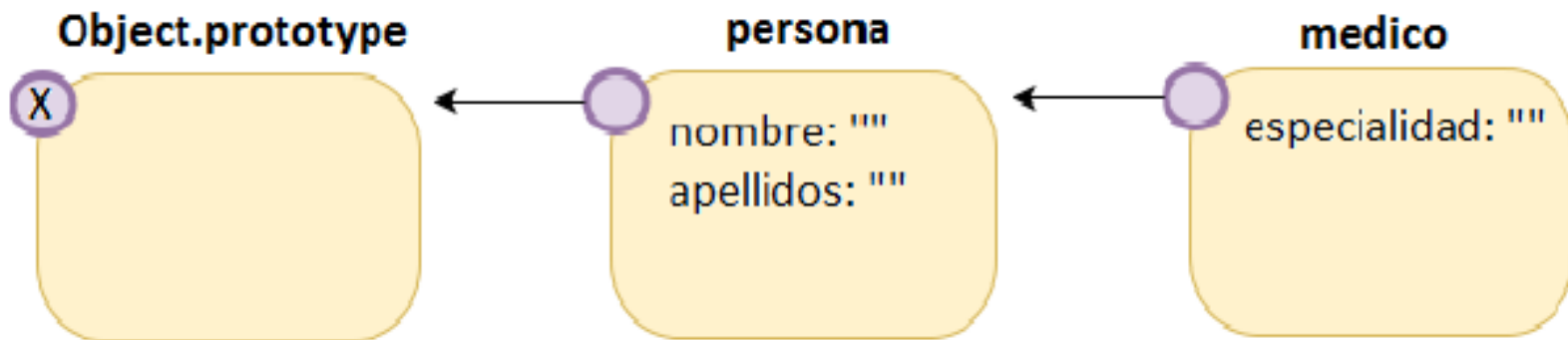
El prototipo de `Object.prototype` es `null`



LA FUNCIÓN Object.getPrototypeOf()

Permite consultar el prototipo de un objeto.

```
let persona = {  
  nombre: "",  
  apellidos: ""  
}  
  
let medico = Object.create(persona);  
medico.especialidad = "";  
  
console.log(Object.getPrototypeOf(medico) === persona); // → true  
console.log(Object.getPrototypeOf(persona) === Object.prototype); // -
```



EJEMPLO: Creación objetos “parecidos” a Instancias

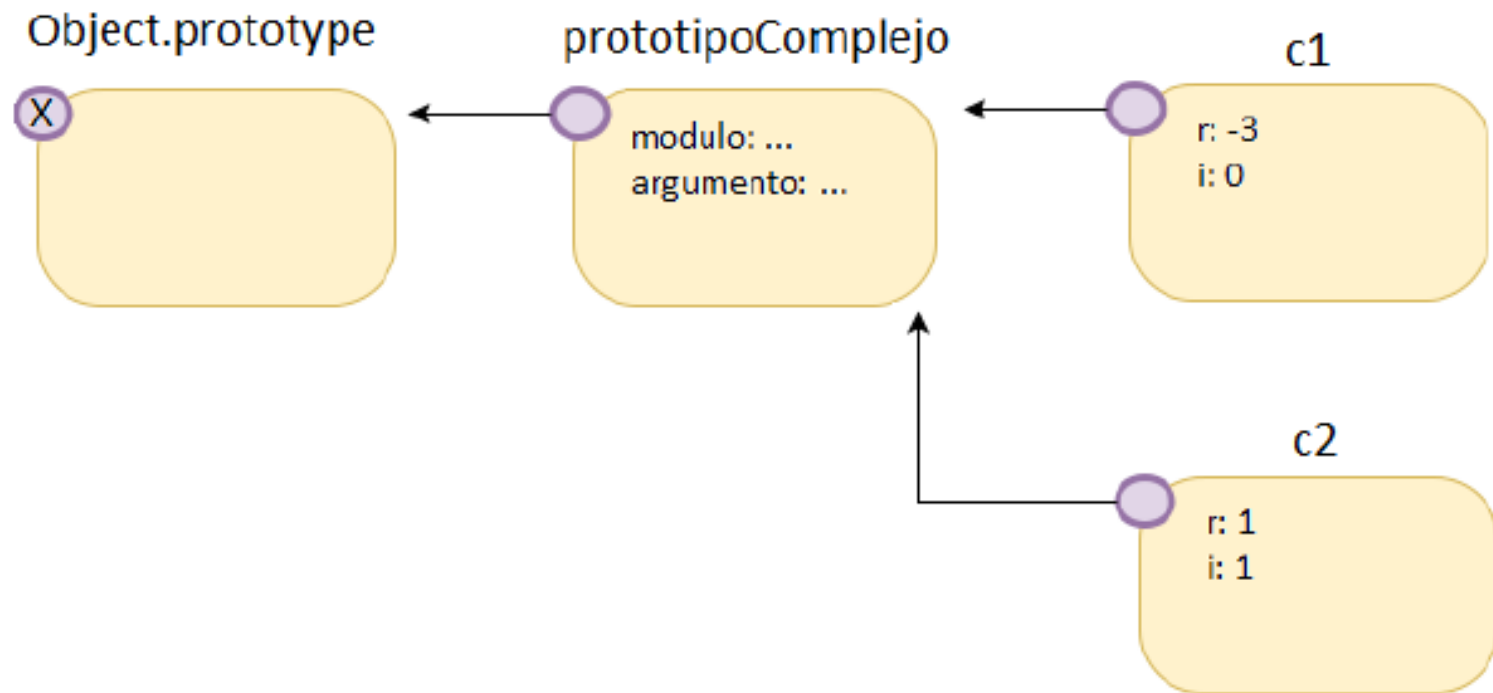
```
// SE CREA UN OBJETO-PROTOTIPO QUE CONTIENE LOS MÉTODOS
let prototipoComplejo = {
  modulo: function() {
    return Math.sqrt(this.r * this.r + this.i * this.i);
  },
  argumento: function() {
    return Math.atan2(this.i, this.r);
  }
};

// SE PROGRAMA UNA FUNCIÓN QUE CREA UN OBJETO A PARTIR DEL PROTOTIPO
// Y LE CREA SUS PROPIOS ATRIBUTOS
function construirComplejo(real, imag) {
  var resultado = Object.create(prototipoComplejo);
  resultado.r = real;
  resultado.i = imag;
  return resultado;
}

var c1 = construirComplejo(-3, 0);
var c2 = construirComplejo(1, 1);
```

EJEMPLO: Creación objetos “parecidos” a Instancias

La cadena de prototipos es la siguiente:



El prototipo tiene los métodos y los objetos los atributos.

EJEMPLO: Creación objetos “parecidos” a Instancias

Todas las funciones/métodos que se añadan al prototipo estarán disponibles automáticamente para todos los objetos que hayan sido creados previamente por `construirComplejo()`.

```
// Añadimos una nueva función al prototipo:

prototipoComplejo.mostrarEnPolares = function() {
  console.log(`(${this.modulo()}, ${this.argumento()})`);
}

c1.mostrarEnPolares();
// → (3, 3.141592653589793)

c2.mostrarEnPolares();
// → (1.4142135623730951, 0.7853981633974483)
```

MÉTODOS DEL OBJETO `Object.prototype`

Como el objeto `Object.prototype` es la raíz de la cadena de prototipos, todos los objetos pueden invocar y sobrescribir sus métodos predefinidos, como por ejemplo:

- `toString()`
- `valueOf()`
- `isPrototypeOf()`
- `hasOwnProperty()`
- `[+]`

EL MÉTODO toString()

Este método es invocado automáticamente cada vez que un objeto es referido en un contexto en el que se espera una cadena.

```
prototipoComplejo.toString = function() {  
    return "(" + this.r + "," + this.i + ")";  
}  
  
let c3 = construirComplejo(1, 3);  
  
alert(c3);           // muestra una ventana con el texto (1,3)
```

También se puede invocar al método de manera explícita.

```
console.log(c3.toString());    // → (1,3)
```

EL MÉTODO `valueOf()`

Este método es invocado automáticamente cada vez que un objeto se encuentra en una posición en la que se espera un valor primitivo (un número, un booleano, etc).

```
let prototipoPersona = {  
  valueOf: function() {  
    return this.edad;  
  }  
};  
  
function construirPersona(nom, ap, e) {  
  var resultado = Object.create(prototipoPersona);  
  resultado.nombre = nom;  
  resultado.apellido = ap;  
  resultado.edad = e;  
  return resultado;  
}  
  
var p1 = construirPersona("Juan", "Gómez", 15);  
var p2 = construirPersona("Ana", "Torres", 18);  
  
console.log(p1+p2);    // → 33
```


EL MÉTODO hasOwnProperty()

Este método permite conocer si un objeto tiene un atributo propio o por el contrario dicho atributo está en la cadena de prototipos.

```
let circulo = {  
  centro: {x: 0, y: 0},  
  radio: 5  
};  
  
let c1 = Object.create(circulo);  
let c2 = Object.create(circulo);  
  
console.log(c1.hasOwnProperty("radio")); // → false  
  
c1.radio = 10;  
console.log(c1.hasOwnProperty("radio")); // → true
```

SINTAXIS DE CREACIÓN DE OBJETOS A PARTIR DE ES6

Antes de ES6, un patrón común para simular las **clases** de otros lenguajes de programación es:

- Tener un objeto prototipo que almacene los métodos de la clase.
- Tener una función que construya las instancias de la clase, enlazándolas con el prototipo.

ES6 incorpora una nueva sintaxis para la creación de objetos, denominada "syntactic sugar" porque asemeja la creación de objetos a los lenguajes de clases pero no cambia el modelo de prototipado.

CLASES (I)

```
class Complejo {  
    constructor(real, imag) {  
        this.r = real;  
        this.i = imag;  
    }  
  
    modulo() {  
        return Math.sqrt(this.r * this.r + this.i * this.i);  
    }  
  
    argumento() {  
        return Math.atan2(this.i, this.r);  
    }  
}
```

Esta declaración crea un objeto llamado `Complejo.prototype` que almacena el `constructor()` y los métodos `modulo()` y `argumento()`.

CLASES (II)

Se crean instancias mediante el operador `new`.

```
let c1 = new Complejo(-3, 0);  
let c2 = new Complejo(1, 1);
```

Esto hace que los objetos `c1` y `c2` tengan a `Complejo.prototype` como prototipo.

```
console.log(c1.modulo()); // → 3  
console.log(c2.modulo()); // → 1.4142135623730951
```

CLASES (III)

Complejo.prototype

modulo:

```
function() {  
  return Math.sqrt(  
    this.r * this.r +  
    this.i * this.i  
  );  
}
```

argumento:

```
function() {  
  return Math.atan2(  
    this.i, this.r  
  );  
}
```

c1

r	: -3
i	: 0

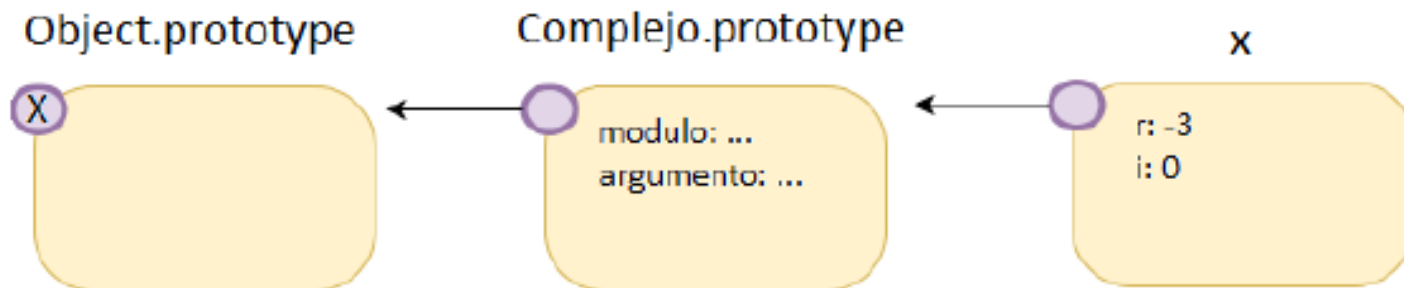
c2

r	: 1
i	: 1

EL OPERADOR instanceof

La expresión `x instanceof C` se evalúa a `true` si el objeto `C.prototype` es alcanzable ascendiendo desde `x` en la cadena de prototipos:

```
var x = new Complejo(-3, 0);  
  
console.log(x instanceof Complejo); // → true  
console.log(x instanceof Object);   // → true  
console.log(x instanceof Number);   // → false
```



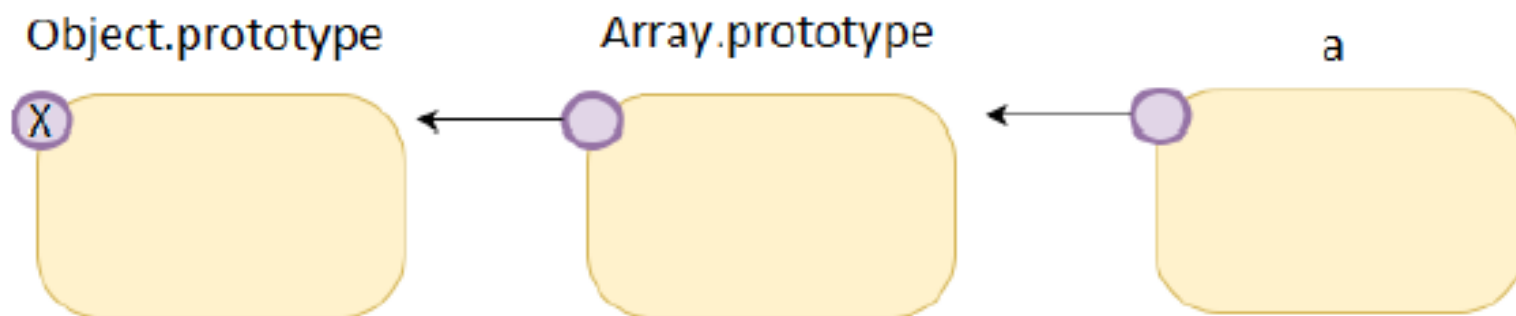
EJEMPLO

```
let a = new Array(3);

console.log(a instanceof Array);           // → true
console.log(a instanceof Object);          // → true

console.log(Object.getPrototypeOf(a) === Array.prototype); // → true
console.log(Object.getPrototypeOf(a) === Object.prototype); // → false

console.log(Array.prototype.isPrototypeOf(a)); // → true
console.log(Object.prototype.isPrototypeOf(a)); // → true
```



FUNCIONES get/set

- Permiten definir métodos cuya invocación es sintácticamente similar a los atributos.
- Permiten asegurar la calidad de los datos.

MÉTODOS INVOCADOS COMO ATRIBUTOS (get)

```
class Complejo {  
  
  constructor(real=1, imag=1) {  
    this.r = real;  
    this.i = imag;  
  }  
  
  modulo() {  
    return Math.sqrt(this.r * this.r  
                      + this.i * this.i)  
  }  
}  
  
let c= new Complejo(1,0);  
console.log(c.modulo()); // → 1
```

```
class Complejo {  
  
  constructor(real=1, imag=1) {  
    this.r = real;  
    this.i = imag;  
  }  
  
  get modulo() {  
    return Math.sqrt(this.r * this.r  
                      + this.i * this.i)  
  }  
}  
  
let c= new Complejo(1,0);  
console.log(c.modulo); // → 1
```

MÉTODOS INVOCADOS COMO ATRIBUTOS (set)

```
class Complejo {  
  
  constructor(real=1, imag=1) {  
    this.r = real;  
    this.i = imag;  
  }  
  
  escala(e) {  
    if (e>0){  
      this.r = this.r*e;  
      this.i = this.i*e;  
    }  
  }  
}  
  
let c= new Complejo(1,0);  
c.escala(2);  
console.log(c); // → {r:2, i:0}
```

```
class Complejo {  
  
  constructor(real=1, imag=1) {  
    this.r = real;  
    this.i = imag;  
  }  
  
  set escala(e) {  
    if (e>0){  
      this.r = this.r*e;  
      this.i = this.i*e;  
    }  
  }  
}  
  
let c= new Complejo(1,0);  
c.escala=2;  
console.log(c); // → {r:2, i:0}
```

CALIDAD DE LOS DATOS: atributos privados I

```
class Persona {  
  _edad;  
  _nombre;  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this._edad = edad;  
  }  
  get edad() {  
    return this._edad;  
  }  
  set edad(newEdad) {  
    if (newEdad >= 0) {  
      this._edad = newEdad;  
    }  
  }  
}
```

accediendo: 23
modificando: 2

```
let p = new Persona("Elena", 23);  
console.log('accediendo: ' + p.edad);  
p.edad = 2; // Modifica la edad  
console.log('modificando: ' + p.edad);
```

Existe un convenio tácito por el que considera que los atributos que comienzan por `_` son "privados".

Estado de la propuesta

```
class Persona {  
  #edad;  
  #nombre;  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this.#edad = edad;  
  }  
  get edad() {  
    return this.#edad;  
  }  
  set edad(newEdad) {  
    if (newEdad >= 0) {  
      this.#edad = newEdad;  
    }  
  }  
}
```

accediendo: 23
modificando: 2

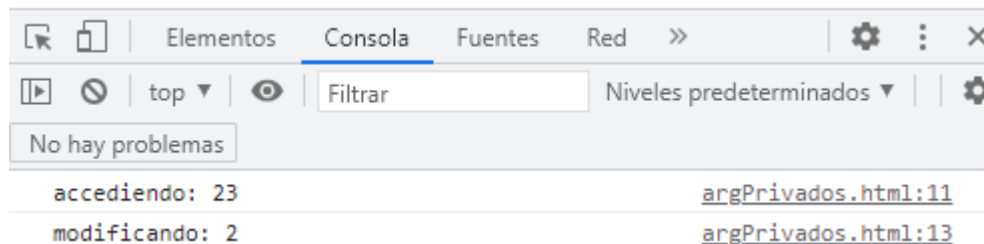
```
let p = new Persona("Elena", 23);  
console.log('accediendo: ' + p.edad);  
p.edad = 2; // Modifica la edad  
console.log('modificando: ' + p.edad);
```

Las últimas versión de Node.js y de navegadores como Chrome ya deberían soportar atributos privados reales anteponiendo el carácter `#` antes del nombre del atributo.

CALIDAD DE LOS DATOS: atributos privados II

```
class Persona {  
  _edad;  
  _nombre;  
  constructor(nombre, edad) {  
    this.nombre = nombre;  
    this._edad = edad;  
  }  
  get edad() {  
    return this._edad;  
  }  
  set edad(newEdad) {  
    if (newEdad >= 0) {  
      this._edad = newEdad;  
    }  
  }  
}
```

```
<!DOCTYPE html>  
<html>  
  <head>  
    <meta charset='utf-8'>  
    <title>Page Title</title>  
    <script src='argPrivados.js'></script>  
  </head>  
  <body>  
    <script>  
      let p = new Persona("Elena", 23);  
      console.log('accediendo: ' +  
p.edad);  
      p.edad = 2; // Modifica la edad  
      console.log('modificando: ' +  
p.edad);  
    </script>  
  </body>  
</html>
```



MÉTODOS ESTÁTICOS

Son métodos de clase. Van precedidos por **static**.

```
class Complejo {  
    // ...  
  
    static desdePolar(mod, arg) {  
        var real = mod * Math.cos(arg),  
            imag = mod * Math.sin(arg);  
        return new Complejo(real, imag);  
    }  
}
```

```
let c2 = Complejo.desdePolar(1, Math.PI / 4);  
console.log(c2.r); // → 0.7071067811865476
```

Es posible añadir métodos estáticos una vez declarada la clase:

```
Complejo.desdePolar = (mod, arg) => {  
    // ...  
}
```

HERENCIA (I)

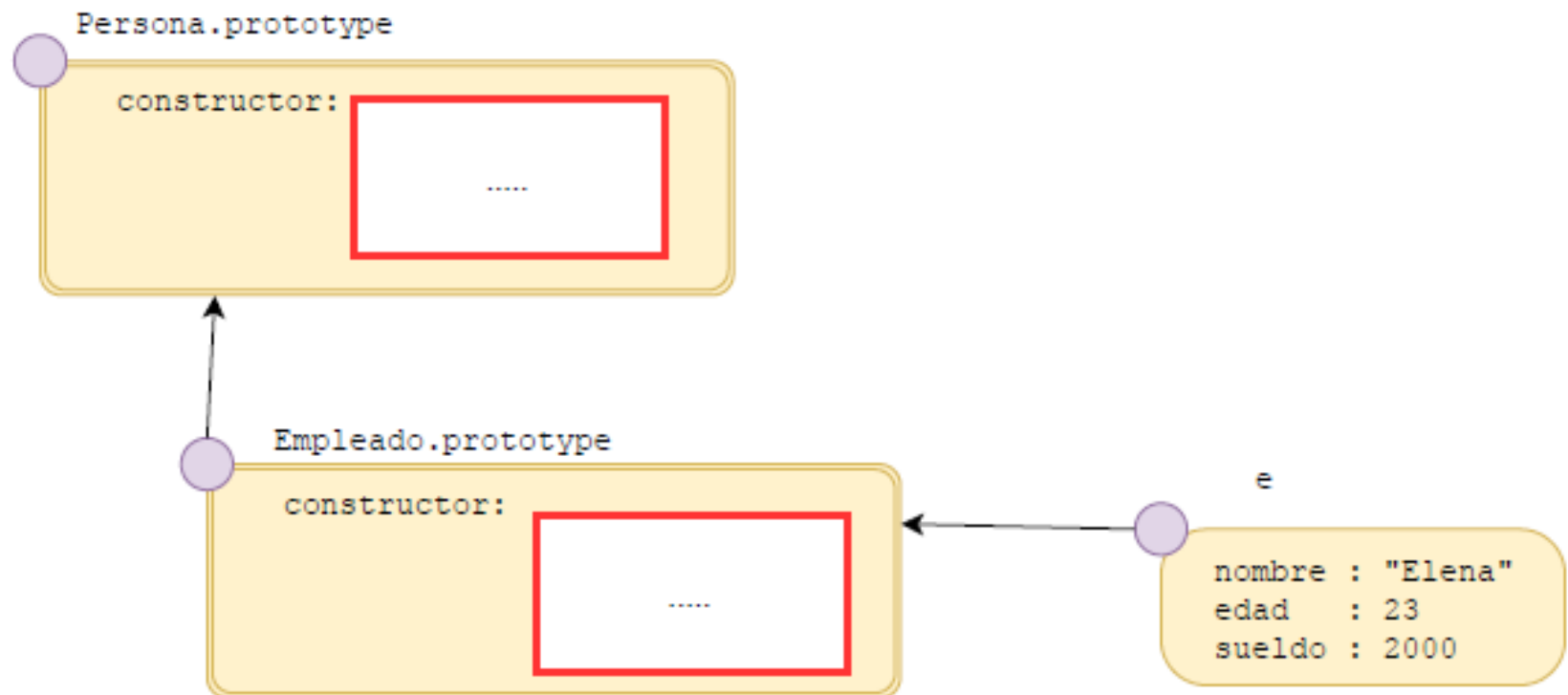
Se permite herencia simple, al igual que en Java.

```
class Persona {  
    constructor(nombre, edad) { ... }  
    ...  
}  
  
class Empleado extends Persona {  
    constructor(nombre, edad, sueldo) {  
        super(nombre, edad);  
        this.sueldo = sueldo;  
    }  
    ...  
}
```

```
let e = new Empleado("Elena", 23, 2000);  
console.log(e.edad);    // → 23  
console.log(e.sueldo);  // → 2000
```

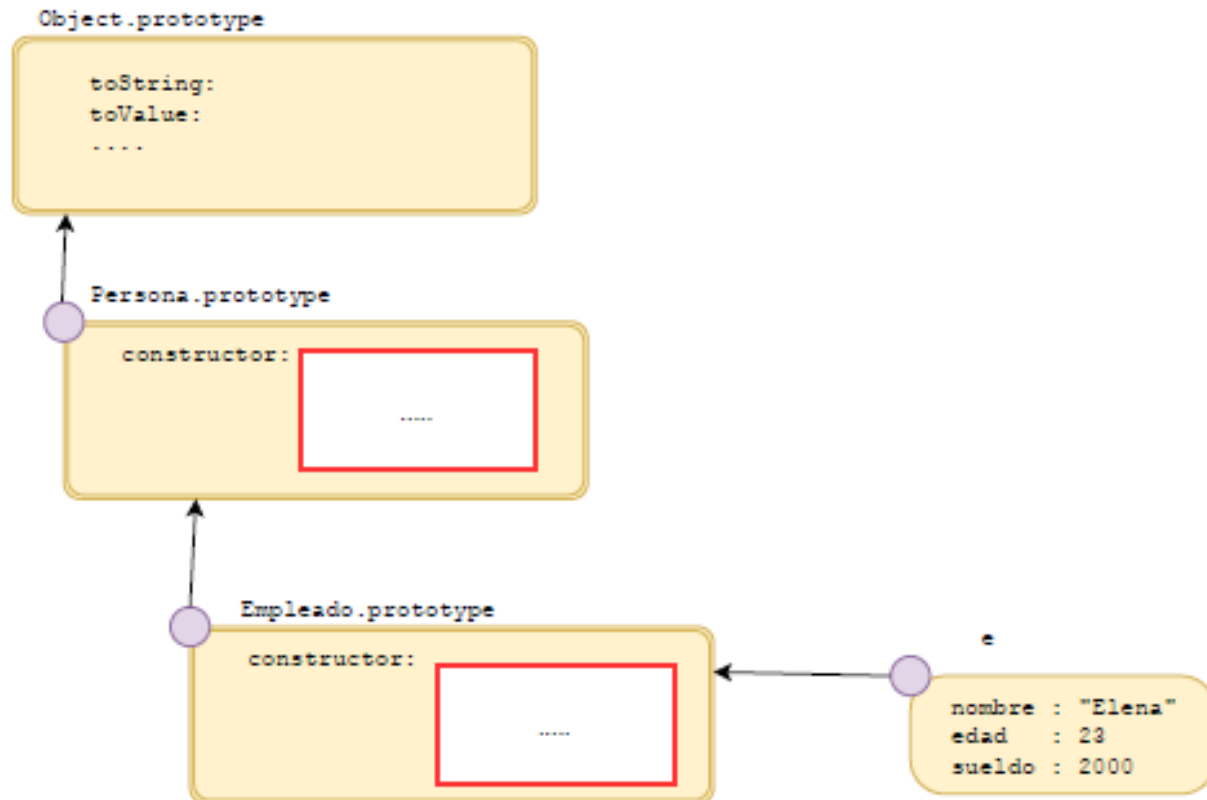
HERENCIA (II)

Al decir que **Empleado** extiende a **Persona**, estamos encadenando el prototipo de persona con el de empleado.



HERENCIA (III)

Si no se indica cláusula **extends**, la clase hereda automáticamente de **Object**



BIBLIOGRAFÍA

- A. Rauschmayer
Speaking Javascript
O'Reilly (2014)
- E. Brown
Learning Javascript, 3rd edition
O'Reilly (2016)
- **Javascript Reference**
MDN - Mozilla Developer Network
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>

